Requirements / Design and Test Documentation (RDD)

Version 1.0

ESEP – Praktikum – Wintersemester 26/27

Team – ESEP-2026WS-3.2

Peters, Emil, 2797681, emil.peters@haw-hamburg.de

Danandeh, Mohammad, 2787662, mohammad.danandeh@haw-hamburg.de

Naghashi, Amirhossein, 2792529, amirhossein.naghashi@haw-hamburg.de

Freiherr von Seld, Alexander, 2797652, alexander.freiherrvonseld@haw-hamburg.de

Änderungshistorie:

Version	Erstellt	Autor	Kommentar
0.1	2018-03-12	LMN	Initiale Version des Templates.
0.2	2020-03-15	DAI	Überarbeitung wegen Corona.
0.3	2022-02-24	LMN	Anpassungen für Sommersemester. Anforderungen an Requirements reduziert auf Ergänzungen.
0.4	2022-11-22 ff.	CHRS	Neustrukturierung des Templates, Schriftgrößen vereinheitlicht, Erweiterungen: Hinweise am Anfang des Dokuments, Unterkapitel Hardware und technische Gegebenheiten, Unterkapitel Analyse des Kundenwunsches, Unterkapitel Nachrichten und Signale, allg. Abnahmetest Text + Tabelle, Unterkapitel Abbildungsverzeichnis
0.5	2023-08-22	CHRS	Korrekturen Rechtschreibung/Ausdruck, Erweiterung der Hilfestellungen, Erweiterung um „Grobkonzept“, Umformulierung und Erweiterung von Texten, Anpassung Titelblatt
0.6	2023-09-18	CHRS	Redaktionelle Bearbeitung
0.7	2023-09-27	CHRS	Bsp. für Test-Tabelle hinzugefügt
0.8 + 0.9	2024-08-28	CHRS	Redaktionelle Bearbeitung
1.0	2026-10-14	AF, AN, EP, MD	Initiale Einigung für Repo Konzept und Aufgabenverteilung und Rahmenbedingungen für das Projekt
2.0	2026-10-14	AF, An, EP, MD	Abgabe Praktikum 1, 04.11.2026

Inhaltsverzeichnis:

1 Teamorganisation 6
1.1 Verantwortlichkeiten 7
1.2 Absprachen 7
1.3 Repository-Konzept SOHRAB 8
2 Projektmanagement EMIL 14
2.1 Prozess 14
2.2 Projektorganisation 14
2.3 Risiken 15
2.4 Qualitätssicherung 17
3 Problemanalyse 18
3.1 Analyse des Kundenwunsches ALEX 18
3.1.1 Stakeholder 20
3.1.2 Systemkontext des Systems 21
3.1.3 Anforderungs Ergänzung 22
3.1.4 Use Cases / User Stories 23
3.2 Anlage: Analyse der technischen Gegebenheiten AMIR 38
3.2.1 Technischer Aufbau und Hardwarekomponenten 38
3.2.2 Werkstücke 40
3.2.3 Anforderungen aus dem Verhalten und technischen Besonderheiten 49
3.3 Softwareebene 51
3.3.1 Systemkontext der Software 51
3.3.2 Resultierende Anforderungen an die Software 51
3.3.3 Schnittstellen: Nachrichten und Signale 51
4 Grobkonzept des technischen Systementwurfes 52
5 Software-Design 53
5.1 Software Architektur 53
5.2 Software Struktur 53
5.3 Verhaltensmodellierung 53
6 Implementierung: Besonderheiten 53
7 Qualitätssicherung 54
7.1 Teststrategie 54
7.2 Testszenarien/Abnahmetest 54
7.3 Testprotokolle und Auswertungen 54
8 Technische Schulden 55
9 Lessons Learned 55
10 Anhang 55
10.1 Glossar 55
10.2 Abkürzungen 55

1 Teamorganisation

In diesem Abschnitt werden Verantwortlichkeiten und Absprachen behandelt. Es wird auch das Repository Konzept kurz vorgestellt. Als Hauptwerte im Team sollen Vertrauen, netter Umgang und lösungsorientiertes Denken in diesem Projekt gelebt werden.

1.1 Verantwortlichkeiten

In der folgenden Tabelle werden die Verantwortlichkeiten im Team den Teammitgliedern zugeordnet.

Verantwortlichkeit	Person/en
Product Owner	Emil Peters
Scrum Master	Alexander Freiherr von Seld
Implementierer	Amirhossein Naghashi, Mohammad Danandeh, Emil Peters, Alexander Freiherr von Seld
Software Quality Assurance	Mohammad Danandeh

1.2 Absprachen

Als Kommunikation innerhalb des Teams wird Microsoft Teams verwendet. Hierüber werden Abwesenheiten oder Ähnliches kommuniziert. Dienstag und Mittwoch stehen als Tage für das gemeinsame Arbeiten vor Ort (neben den Vorlesungen) zur Verfügung. Dort werden die aktuellen Arbeitsstände der einzelnen Teilaufgaben zusammengetragen und gemeinsam die neuen Aufgaben für die nächste Woche definiert und zugeteilt.

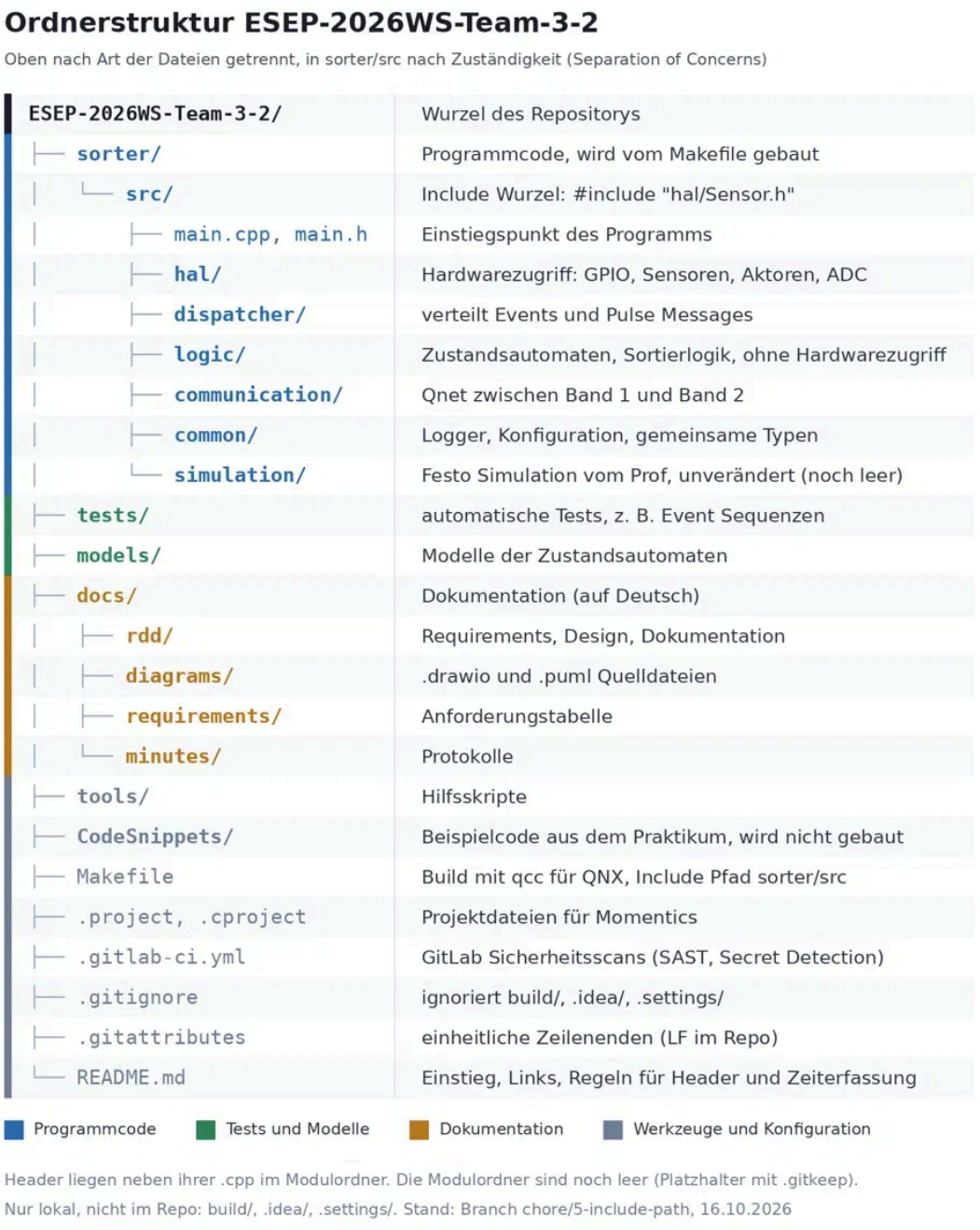
1.3 Repository-Konzept

In diesem Kapitel wird das Repository-konzept und die Ordner Struktur festgelegt und anschließend spezifiziert. Zusätzlich werden die Regeln für den Umgang mit Branches, Auslieferungen, Nachrichten an den Commits ergänzt und wie die Arbeitsabläufe sowie die Umsetzung von Arbeitsaufträgen bei der Behebung von Fehlern im Zusammenhang mit dem Repository beschrieben.

Repository und Ordner: Der Code liegt im GitLab Projekt ESEP-2026WS-Team-3-2 der HAW.

- Auf oberster Ebene ist das Repository nach Art der Dateien getrennt:
  - sorter/src enthält den Programmcode.
  - tests
  - Tools enthält Hilfs Skripte.
  - models enthält die Modelle der Zustandsautomaten für Itemis Create
  - docs enthält die Dokumentation mit den Unterordnern rdd, diagrams, requirements und minutes.
  - Code Snippets enthält Beispielcode aus dem Praktikum, der nicht mitgebaut wird.
- Innerhalb von sorter/src gibt es einen Ordner pro **Komponente**.
- Header liegen neben ihrer .cpp Datei im jeweiligen Modul Ordner, einen eigenen include Ordner gibt es nicht.

- Das Makefile baut das Programm mit qcc für QNX. Die Projektdateien von Momentics (.project, .cproject) liegen ebenfalls im Repository, damit jeder das Projekt direkt importieren und mit denselben Einstellungen bauen kann.
- Build Ausgaben und persönliche IDE Einstellungen (build, .idea, .settings, .metadata) schließen wir per .gitignore aus. Sie sehen auf jedem Rechner anders aus und würden sonst ständig Konflikte erzeugen.
- Code, Kommentare im Code, Branchnamen und Merge Request Titel schreiben wir auf Englisch, die Dokumentation auf Deutsch. Abgegebene Dokumente bekommen das Präfix ESEP-2026WS-Team-3-2_, z.B. ESEP-2026WS-Team-3-2_RDD.pdf, so wie es die Aufgabe verlangt.



Ordnerstruktur ESEP-2026WS-Team-3-2

Oben nach Art der Dateien getrennt, in sorter/src nach Zuständigkeit (Separation of Concerns)

ESEP-2026WS-Team-3-2/	Wurzel des Repositorys
├── sorter/	Programmcode, wird vom Makefile gebaut
│ └── src/	Include Wurzel: #include "hal/Sensor.h"
│ ├── main.cpp, main.h	Einstiegspunkt des Programms
│ ├── hal/	Hardwarezugriff: GPIO, Sensoren, Aktoren, ADC
│ ├── dispatcher/	verteilt Events und Pulse Messages
│ ├── logic/	Zustandsautomaten, Sortierlogik, ohne Hardwarezugriff
│ ├── communication/	Qnet zwischen Band 1 und Band 2
│ ├── common/	Logger, Konfiguration, gemeinsame Typen
│ └── simulation/	Festo Simulation vom Prof, unverändert (noch leer)
├── tests/	automatische Tests, z. B. Event Sequenzen
├── models/	Modelle der Zustandsautomaten
├── docs/	Dokumentation (auf Deutsch)
│ ├── rdd/	Requirements, Design, Dokumentation
│ ├── diagrams/	.drawio und .puml Quelldateien
│ ├── requirements/	Anforderungstabelle
│ └── minutes/	Protokolle
├── tools/	Hilfsskripte
├── CodeSnippets/	Beispielcode aus dem Praktikum, wird nicht gebaut
├── Makefile	Build mit qcc für QNX, Include Pfad sorter/src
├── .project, .cproject	Projektdateien für Momentics
├── .gitlab-ci.yml	GitLab Sicherheitsscans (SAST, Secret Detection)
├── .gitignore	ignoriert build/, .idea/, .settings/
├── .gitattributes	einheitliche Zeilenenden (LF im Repo)
└── README.md	Einstieg, Links, Regeln für Header und Zeiterfassung

Programmcode | Tests und Modelle | Dokumentation | Werkzeuge und Konfiguration

Header liegen neben ihrer .cpp im Modulordner. Die Modulordner sind noch leer (Platzhalter mit .gitkeep).
Nur lokal, nicht im Repo: build/, .idea/, .settings/. Stand: Branch chore/5-include-path, 16.10.2026

Abbildung 1.3.1: Ordnerstruktur

Branches

- Der Main-Branch ist geschützt: Niemand pusht direkt darauf, Änderungen kommen nur über Merge Requests hinein. So bleibt die Main-Branch jederzeit lauffähig.
- Für jedes Issue legen wir einen kurzlebigen Branch an, der direkt von dem Main-Branch abzweigt.
    - Er heißt typ/issuenummer-kurzbeschreibung, z.B. feature/12-measure-height oder fix/15-sensor-timeout. Mögliche Typen sind feature, fix, refactor, test, docs und chores.
    - Am Namen ist sofort erkennbar, zu welchem Issue ein Branch gehört und welche Art von Änderung er enthält.
    - Weil ein Branch nur wenige Tage lebt, laufen die Stände nicht weit auseinander, und Merge Konflikte bleiben klein.
- Nach dem Merge löscht GitLab den Branch automatisch. So zeigt die Branchliste nur die Arbeit, die gerade läuft.

Commit Nachrichten und Merge

- Titel von Merge Requests folgen dem Schema typ: kurzbeschreibung, z.B. feature: read height values from sensor oder fix: handle sensor timeout. Die Typen sind dieselben wie bei den Branches. Durch das einheitliche Vokabular bleibt die Historie auf dem Main-Branch lesbar, und man erkennt schon an der Nachricht, was eine Änderung bewirkt.
- Gemergt wird per Squash. Dabei werden alle Commits eines Merge Requests zu einem einzigen zusammengefasst, dessen Nachricht der Titel des Merge Requests ist. Deshalb muss nur dieser Titel dem Schema folgen. *Die Commits auf dem Branch dürfen frei benannt werden.*

Merge Requests und Review

- Jeder Merge Request braucht die Freigabe (Review approval) eines anderen Teammitglieds. Danach merged die Person, die ihn geöffnet hat. Durch das Review fallen Fehler früher auf, und jeder lernt auch den Code der anderen kennen. Reviews erledigen wir innerhalb eines Werktags, damit niemand blockiert ist.
- In GitLab ist eine Vorlage für die Beschreibung in die Merge Request- Einstellungen hinterlegt. Unter Warum stehen ein kurzer Grund für die Änderung und Closes #issuenummer. Dadurch schließt sich das zugehörige Issue bei dem Merge mit dem Main-Branch automatisch.
- Die Checkliste unter Erledigt ist unsere Definition of Done:
    - kompiliert ohne Warnungen
    - Tests laufen erfolgreich
    - an der Anlage ausprobiert, falls Hardware betroffen ist
    - Doku angepasst (RDD, Diagramme, README), falls betroffen
    - Testergebnis im Merge Request oder im Testprotokoll festgehalten
    - Zeiterfassung für die Issue
- Da diese Liste in jedem Merge Request steht, prüfen alle gegen dieselben Kriterien, und Tests an der echten Anlage sind auch im Nachhinein nachvollziehbar.
- Die GitLab Pipeline führt nur automatische Sicherheits Scans aus. Sie ist keine Bedingung für den Merge, damit kein zusätzlicher Aufwand entsteht.

Auslieferungen

- Jeder Stand, den wir an einem Praktikumstermin vorführen, bekommt einen annotierten Tag auf main : termin-2 bis termin-5 und endabnahme für den 6. Termin. Getaggt wird nur ein Stand, der an der Anlage getestet wurde. So haben wir den vorgeführten Stand sicher, auch wenn main kurz vorher kaputt geht, und können belegen, was zu welchem Termin lief. Die Sprints bilden wir über Meilensteine in GitLab ab.
- Tags werden nie verschoben oder gelöscht. Eine spätere Korrektur bekommt einen eigenen Tag, z.B. termin-1-fix.

Arbeitsablauf bei Aufgaben

1. Aufgabe als Issue anlegen und mit /estimate schätzen.
2. Aktuellen Stand von main holen, Branch anlegen und umsetzen.
3. Mindestens einmal am Tag den neuen Stand von main per Rebase übernehmen, damit Konflikte früh und klein auftreten.
4. Merge Request mit Closes #issuenummer öffnen und eine Person als Reviewer eintragen.
5. Review, Korrekturen auf derselben Branch.
6. Squash Merge auf den Main-Branch , danach den lokalen Branch löschen.

Braucht eine Aufgabe etwas aus einer anderen, wird die erste zuerst gemergt. Änderungen an gemeinsamen Headern, Schnittstellen oder am Build kündigen wir vorher in der Gruppe an und mergen sie als eigenen kleinen Merge Request. Bei einem Konflikt löst ihn, wer später mergt.

Arbeitsablauf bei Fehlern

Fehler laufen genauso ab, mit drei Unterschieden:

- Das Issue bekommt das Label bug und beschreibt, wie man den Fehler auslöst. So lässt sich beim Review prüfen, ob der Fix wirkt.
- Der Branch heißt fix/issuenummer-kurzbeschreibung.
- Betrifft der Fehler einen getaggten Stand, kommt der Fix normal auf main und bekommt danach einen neuen Tag. Der alte Tag bleibt unverändert, damit nachvollziehbar ist, was zum damaligen Termin vorgeführt wurde.

Zeiterfassung

- Schätzung und gearbeitete Zeit buchen wir per Quick Action als Kommentar am Issue, z.B. /estimate 4h und /spend 1h30m. So hängen die Zeiten an der Aufgabe statt am Code, und GitLab summiert sie automatisch.
- Wir buchen nur in Stunden und Minuten. Einheiten wie 1D vermeiden wir, weil GitLab einen Tag als 8 Stunden berechnet.
- Für gemeinsame Meetings und Aufgaben zu Hause legen wir eine Task mit dd.MM_ am Anfang des Titels an, unter der jeder Teilnehmer seine Zeit selbst bucht. Zu Beginn jedes Praktikumstermins müssen alle Zeiten gebucht sein, so verlangt es die Aufgabe.

2 Projektmanagement

Es wurde sich auf die SCRUM - Methode als Framework für die Teamarbeit geeinigt. Dadurch soll schnell auf Veränderungen im Projektverlauf reagiert werden können. Dieser Prozess wird im folgenden Abschnitt erläutert.

2.1 Prozess

Es werden wöchentliche Meetings geplant und gehalten. Dabei werden die zu bearbeitenden Anforderungen in konkrete Issues zerlegt (falls noch nicht geschehen) und im Team verteilt. Als Meilensteine wird sich an den Anforderungen des Praktikums orientiert. Außerdem wird das Feedback über die letzte Woche besprochen. Hierbei können ebenfalls die Anforderungen angepasst werden.

2.2 Projektorganisation

Die Struktur dieses Projektes wird komplett über Git abgebildet. Die Meilensteine beschreiben die Anforderungen aus der Aufgabenstellung für die jeweiligen Praktika. Die Anforderungen der Aufgabenstellung wurden hierfür in mehrere Epics zerteilt, die nur über die “group” zu erreichen sind.

Die Epics werden wiederum in kleine Issues (welche wiederum in Tasks zerteilt werden können) zerstückelt, die zu bearbeiten sind. Hierbei werden alle Issues, welche sich auf Funktionalitäten der Software Steuerung beziehen, als User-Stories abgebildet. Sie werden mit dem Präfix *USx_Name* eingestellt. Die Anforderungen an Dokumentation und Planung werden separat als Issues mit dem Präfix der jeweiligen Praktikumsanforderung mit *Px_Name* eingestellt.

Eine Liste der gesamten Aufgaben (Project Backlog) befindet sich also im Issue Board des Projektes. Dabei werden alle Aufgaben, welche aktiv bearbeitet werden müssen, für den aktuellen Sprint (Sprint Log) unter dem Reiter “in progress” gesetzt.

Die Ressourcen werden in den wöchentlichen Meetings verteilt. So kann flexibel auf Aufwandsänderungen eingegangen werden.

Als zeitliche Orientierung dient folgender Zeitstrahl. Dieser beschreibt die Meilensteine welche jeweils montags vor dem Abgabetermin erfüllt sein müssen. Die Aufgaben zu diesen Meilensteinen sind jeweils auf Git zu finden.

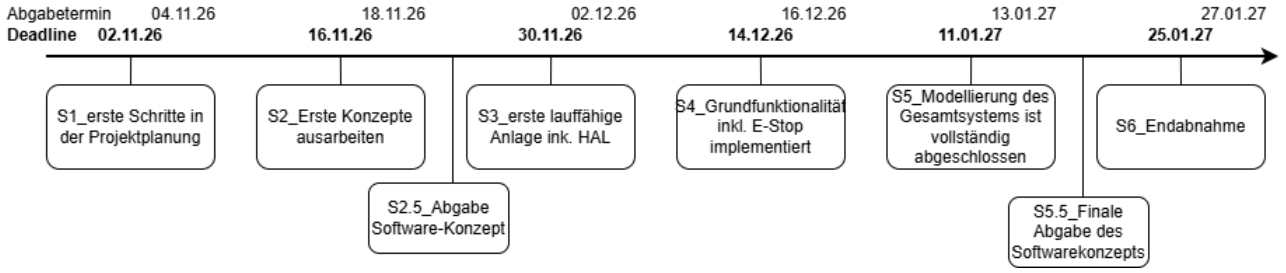


Abbildung 2.2.1: Zeitstrahl Projektorganisation

Um das Risiko von Zeitverzug zu minimieren, ist die jeweilige Deadline Montags vor der Abgabe, wo gemeinsam die bisherigen Ergebnisse durchgegangen werden. So soll gewährleistet werden, dass die Anforderungen komplett und mit ausreichender Qualität erfüllt werden.

2.3 Risiken

Wahrscheinlichkeiten werden unterteilt in:

1 unwahrscheinlich
2 denkbar
3 wahrscheinlich

Schadensklassen werden unterteilt in:

1 geringer Schaden
2 beträchtlicher Schaden
3 großer Schaden

Risikobeschreibung	Wahrscheinlichkeit	Schaden	Wert(WxS)	Maßnahme
Krankheitsfall eines Teammitglieds	wahrscheinlich	geringer Schaden	3x1=3	Frühzeitige Kommunikation, sodass alle Teammitglieder bescheid wissen, gepflegter Backlog sodass Aufgaben klar sind und evtl. übernommen werden können
keine Verfügbare Anlage (Hardware) zum arbeiten	denkbar	beträchtlicher Schaden	2x2=4	Arbeitseinteilung sodass zu Stoßzeiten auch andere Aufgaben erledigt werden können, Nutzung mit der Simulation
Abbruch eines Teammitglieds	unwahrscheinlich	großer Schaden	1x3=3	Netter Umgang miteinander, gegenseitige Rücksicht nehmen auch in stressigen Situationen, gemeinsam Lösung erarbeiten statt “finger pointing”
Issues nehmen mehr Zeit in Anspruch als anfangs angenommen	wahrscheinlich	betr ächtliche r Schaden	3x2=6	Zeitpuffer einbauen wo möglich, falls andere Ressourcen frei haben evtl parallele Aufgaben abnehmen
technische Probleme eines oder mehrerer Teammitglieder	denkbar	geringer Schaden	2x1=2	Ansprechpartner für Themen bestimmen, schwierige / komplizierte Einrichtungen dokumentieren

2.4 Qualitätssicherung

Um Die Qualität in diesem Projekt sicherzustellen gibt es mehrere Ebenen mit jeweiligen Maßnahmen:

1. Ebene - Code und Implementierung

- Sauberer und verständlicher Code: Der Code wird nachvollziehbar strukturiert und kommentiert, um die Wartbarkeit und Lesbarkeit durch alle Teammitglieder unter Einhaltung gemeinsamer Standards sicherzustellen.
- Komponententests: Vor jeder Abgabe oder Zusammenführung werden die jeweiligen Teilabschnitte isoliert getestet.
- Git-Merge-Regeln: Es gelten verbindliche Richtlinien für die Arbeit mit Git. (vgl. Kapitel 1.3)

2. Ebene - System, Hardware und Schnittstellen

- Getrennte Tests: Hardware und Software-Logik werden zunächst unabhängig voneinander geprüft (optional unter Verwendung von Mocks), um funktionierende Teilsysteme zu gewährleisten.
- FSM-Validierung: Kritische Szenarien und die Funktionsweise der Finite State Machine (FSM) werden anhand von Werkzeugen (z. B. itemis CREATE) durchgespielt und abgesichert.

3. Ebene - Dokumentation, Spezifikation und Modelle

- Gemeinsame Reviews: Regelmäßige Team-Durchsichten von Spezifikationen, Modellen und Dokumentationen zur frühzeitigen Minimierung von Fehlern und Schwachstellen.

4. Ebene - Prozess und Teamqualität

- Regelmäßige Abstimmungen: Durch kontinuierliche Treffen und prozessbezogene Gespräche wird die Qualität der Zusammenarbeit gesteuert, um auf Probleme schnell reagieren zu können.

3 Problemanalyse

In diesem Kapitel werden die Anforderungen des Kunden analysiert und anschließend spezifiziert. Zusätzlich wurden die CBMs anhand der bereitgestellten Dokumentationen sowie durch eine Vor-Ort-Analyse und Auswertung der Beobachtungen untersucht.

3.1 Analyse des Kundenwunsches

Im folgenden Abschnitt analysieren wir die Anforderungen und arbeiten mögliche Überschneidungen oder Konflikte heraus. Zur Veranschaulichung und schematischen Darstellung der Anlage wurde die nachfolgende Grafik erstellt, worauf wir uns in der Analyse beziehen.

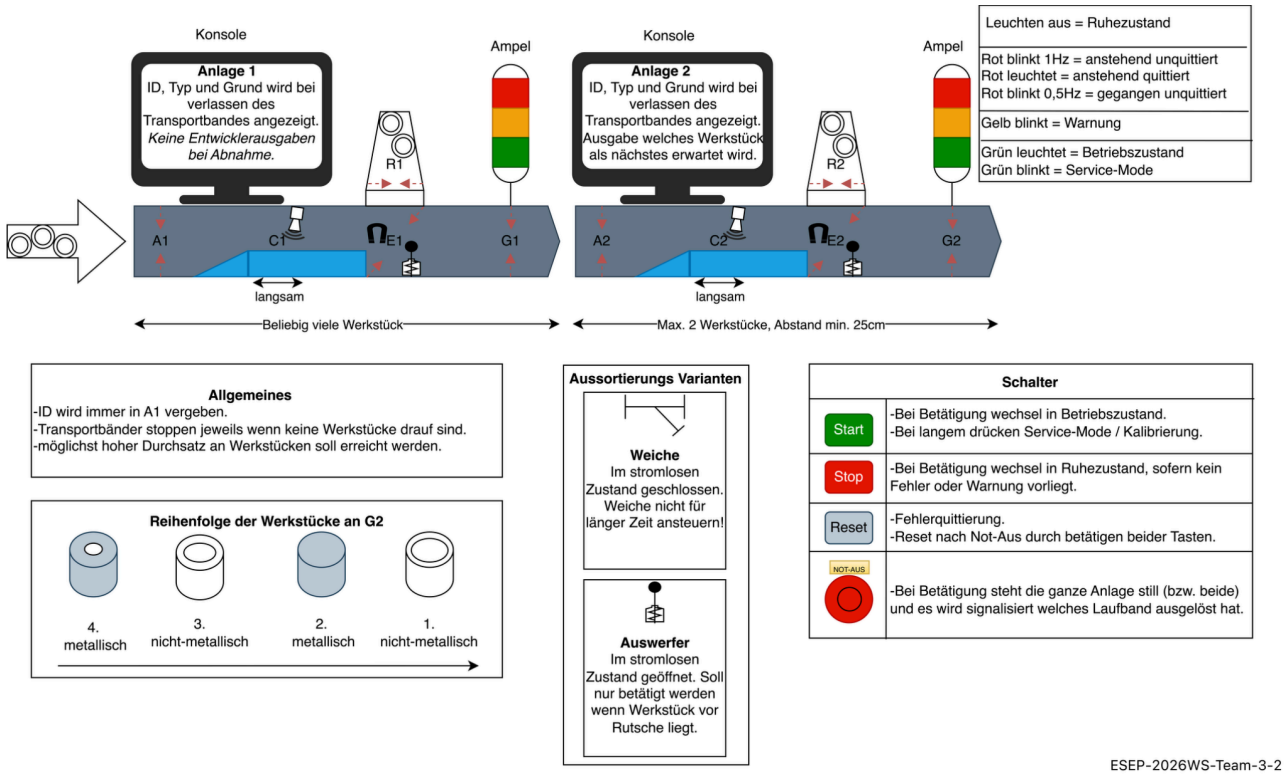


Abbildung 3.1.1: Darstellung des Systems (Quelle: eigene Darstellung)

Die Anlage soll aus zwei gekoppelten Festo Conveyor Belt Modules (CBM) bestehen, die jeweils über einen eigenen Steuerungsrechner (Beagle Bone Black) gesteuert werden. Die Kommunikation zwischen den beiden Steuerungsrechnern erfolgt über Ethernet unter Verwendung des Qnet-Protokolls.

Materialfluss, Sensorik und Aktorik

- Zuführung der Werkstück und ID vergabe:

  Wie in der Grafik oben unter “Allgemeines” und am linken Einlauf dargestellt, erfolgt die Zuführung der Werkstücke manuell am Anfang des Conveyor Belt Modules 1 (A1). Sobald ein Werkstück eingelegt wird, vergibt das System dem Werkstück eine ID. Die Transportbänder stoppen im Leerlauf automatisch, wenn sich keine Werkstücke auf ihnen befinden. Auf Conveyor Belt Module 1 dürfen sich, im Gegensatz zu Conveyor Belt Module 2, beliebig viele Werkstücke gleichzeitig befinden.

- Höhenmessung und Sensoren:

  Beide CBMs leiten die Werkstücke langsam durch den Messbereich der Höhenmessung (C1/C2). Der Laser der Höhenmessung ist dabei leicht verkippt, um eine Eigenblendung des Sensors zu verhindern.

- Aussortierung und Varianten:

  Wenn die Rutsche des 1. Conveyor Belt Modules voll ist, werden Fehlerhafte Werkstücke zunächst von Conveyor Belt Module 2 aussortiert. Sollten beide Rutschen voll sein, ist dies als Fehler zu behandeln.

  Wie im Kasten “Aussortierungs Varianten” der Grafik veranschlaulicht, unterscheidet sich die Aktorik zwischen den Anlagentypen wie folgt:

  - ➔ **Weiche:** Im Stromlosen Zustand geschlossen, wodurch Bausteine automatisch in der Rutsche landen. Sie darf nicht für längere Zeit angesteuert werden, da sonst die Magnetspule überhitzt und beschädigt wird.
  - ➔ **Auswerfer:** Im Stromlosen Zustand geöffnet, somit können Bausteine passieren und wird aktiv betätigt, um ein Werkstück in die Rutsche zu drücken.
- Übergabe an Conveyor Belt Module 2:

  Die Übergabe von Conveyor Belt Module 1 an Conveyor Belt Module 2 muss vereinzelt erfolgen. Wie an dem Pfeil in der Grafik unterhalb des Conveyor Belt Modules 2 spezifiziert, dürfen sich auf dem Transportband, im Gegensatz zu Conveyor Belt Module 1, nur maximal 2 Werkstücke mit einem Mindestabstand von 25 cm befinden. Am Ende von Conveyor Belt Module 2 (G2) müssen die Werkstücke in einer exakten, vordefinierten Reihenfolge ankommen (siehe Schema unten link in oberer Grafik).

Bedienung und Sicherheit

- Bedienpanel und Taster:

  Die Funktionen der Taster entsprechen exakt der Übersicht im Kasten “Schalter” der oberen Grafik:

  - ➔ **Start:** Versetzt die Anlage bei Betätigung in den Betriebszustand und ein langes Drücken aktiviert den Service-Mode.
  - ➔ **Stop:** Wechselt in den Ruhezustand, vorausgesetzt es liegen keine Fehler oder Warnungen vor.
  - ➔ **Reset:** Dient der Fehlerquittierung und ist nach einem E-Stop zwingend durch die Betätigung der Taste an beiden CBMs auszuführen.
- E-Stop:

  Wird der rote E-Stop Schalter gedrückt, steht die gesamte Anlage (beide Transportbänder) sofort still. Das entsprechende Conveyor Belt Module signalisiert dabei optisch, an welchem Laufband der E-STOP ausgelöst wurde. Nach dem Entriegeln läuft die Anlage erst dann weiter, wenn der Reset-Taster an Beiden CBMs betätigt wurde.

- Die Ampeln:

  Die LED-Zustände der Ampeln sind direkt rechs oben in der Grafik dokumentiert:

  ➔ **Leuchten aus:** Ruhezustand.
  ➔ **Rot schnelles Blinken (1 Hz):** Fehler anstehen, unquittiert.
  ➔ **Rot leuchtet:** Fehler anstehen, quittiert.
  ➔ **Rot langsames Blinken (0,5 Hz):** Fehler gegangen, unquittiert.
  ➔ **Gelb blinkt:** Warnung liegt vor.
  ➔ **Grün leuchtet:** Normaler Betriebszustand.
  ➔ **Grün blinkt:** Service-Mode aktiv.

3.1.1 Konflikte und Unklarheiten

Nr.	Kundenaussagen	Konflikt/Unklarheit
1	93-98	In welcher Reihenfolge wird quittiert und der Fehler behoben? Wird zunächst der Fehler behoben und anschließend der Fehler quittiert und die Anlage wechselt direkt in den Betriebszustand oder wird der Fehler quittiert, dann der Fehler behoben und anschließend nochmal quittiert/ start gedrückt, um in den Betriebszustand zu wechseln?
2	92	Die Ampel soll Gelb blinken bei Warnung. Wann tritt eine Warnung auf bzw. wie ist diese definiert?
3	10-11	Was ist ein defektes Werkstück und wie ist dieses mit einem Sonderform Werkstück zu unterscheiden.
4	36-38	Stimmt eine Kombination aus erkannten Merkmalen nicht mit den vorgegebenen Typen überein, wie wird dieses Werkstück eingeordnet?
5	45	Wie sieht der genaue Ablauf aus, wenn ein Werkstück mit Sonderform (nicht aussortierbar) erkannt wurde.
6		Hierarchie der Fehler: wenn eine CBM in einem Fehlerzustand ist und ein weiterer Fehler kommt, wie soll das gehandhabt werden?
7		Wird das System nach einem E-Stop komplett zurückgesetzt, sprich alle Werkstücke von den Laufbändern nehmen und die Reihenfolge von Anfang starten?
8	30	Braucht man einen Mindestabstand auf CBM1? Was heißt vereinzelnd auflegen?
9		Was soll bei einem Qnet Verbindungsabbruch passieren?

3.1.2 Anforderungs Ergänzung

Dieser Abschnitt spezifiziert Ergänzungen, Klarstellungen und Verfeinerungen zu den Kundenanforderungen. Der Schwerpunkt liegt auf der Eindeutigkeit von Fehlerzuständen, Bedienfolgen und Prozessausnahmen.

Lfd. Nr. / ID	**Beschreibung**
1	Für das Quittieren von Fehlern haben wurde sich auf folgendes Konzept geeinigt. Wenn ein Fehler, egal ob *anstehend unquittiert* oder *gegangen unquittiert* anliegt, wird dies zunächst dem Bediener über die rote Ampel durch Blinken signalisiert. Anschließend quittiert der Bediener den Fehler über die Reset Taste und die rote Lampe leuchtet durchgehend. Nun behebt der Bediener den Fehler (wenn er nicht schon gegangen ist) und versetzt durch Drücken der Start-Taste das System wieder in den Betriebszustand.
2	Als Warnung haben wir eine betätigte Weiche, welche dem Bediener nicht sofort angezeigt wird. Sollte der Bediener jedoch versuchen, das Conveyor Belt mit der Stop-Taste zu stoppen, funktioniert dies nicht. Die Orange Lampe soll dabei blinken.
3+4	Wir Kategorisieren alle Werkstücke, die auf das Conveyor Belt gelegt werden in insgesamt Sechs, für den Betrieb des Systems relevante Werkstück-Typen. Die 4 Typen, welche die Reihenfolge festlegen, Sonderform-Werkstück, und falsches Werkstück, was sowohl diverse andere Typen, defekte und unbekannte Werkstücke beschreibt.
5	Ein Sonderform Werkstück hat eine so sonderliche Form, dass es über beide Aussortierungs Varianten nicht zuverlässig aussortiert werden kann. Daher ist ein stoppen und manuelles Eingreifen zwangsläufig notwendig. Es wird über einen Fehlerzustand realisiert.
6	Es gibt zwei Hierarchien: kritische Fehler (E-Stop und der Kommunikationsverlust zwischen den Anlagen) sind höher als die Fehler der CBMs. Der Fehler einer CBM muss für einen höheren Fehler verlassen werden.
7	Nach einem E-Stopp sollen alle Werkstücke/Reihenfolge zurückgesetzt werden
8*	Da ein Mindestabstand der Werkstücke auf Conveyor Belt Module 1 auch sinnvoll ist, um eine eindeutige Höhenmessung und Aussortierung durchführen zu können, wollen wir dies als Hinweis über die Leuchte Q1 an dem Bedienfeld realisieren. Somit leuchtet die Leuchte Q1 immer dann, wenn der Bandanfang von CBM 1 für ein neues Werkstück freigegeben ist.
9	Sollte die Verbindung über Qnet der beiden CBMs unterbrochen werden, löst dies auf beiden Modulen eine - E-Stop ähnliche Fehlerbehandlung aus, da der sichere Betrieb des Systems nicht gewährleistet werden kann.

3.1.3 Stakeholder

Für dieses Projekt haben wir folgende Stakeholder definiert.

Stakeholder	Interessen
Teammitglieder	Erfüllung aller Anforderungen, reibungslose Teamkoordination, erfolgreicher Abschluss des Praktikums
Auftraggeber (Betreuer)	Sichere und funktionierende Software, pünktliche Abgabe der Zwischenstände
Bediener	Sicherer, intuitiver und fehlerfreier Betrieb, klare Meldungen auf der Konsole und über die Ampel, Exakte einhaltung der Reihenfolge der Werkstücke
Betreiber	

3.1.4 Systemkontext des Systems

Um die Anforderungen an das System besser spezifizieren zu können, wurde im folgenden Use Case Diagramm der Systemkontext dargestellt.

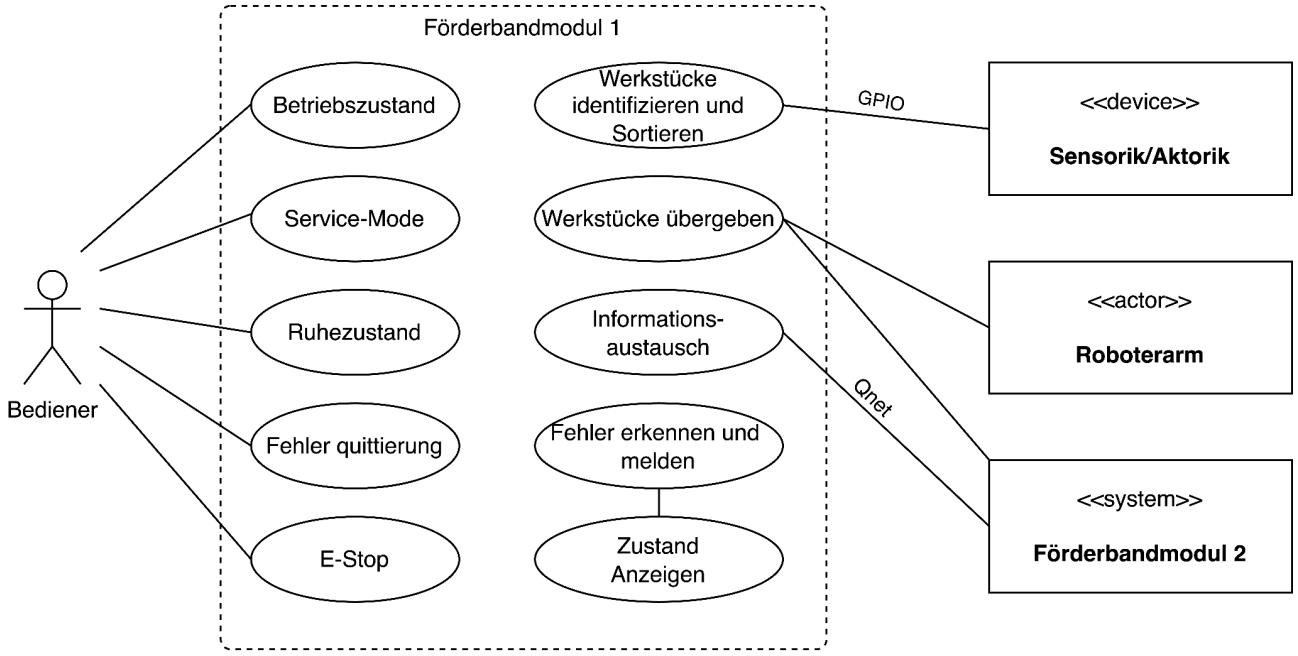


Abbildung 3.1.4.1: UML Use Case Diagramm zum Systemkontext

Der Systemkontext besteht im Wesentlichen aus dem Conveyor Belt Module 1, welches das System als Blackbox darstellt. Der Bediener bedient diverse Funktionen von außen über die Tasten am Bedienfeld und wechselt so den Zustand des Systems. Das System greift über die GPIOs auf die Sensoren und Aktoren als externes Gerät zu, um die Werkstücke zu identifizieren und sortieren zu können. Das Conveyor Belt Module 1 übergibt zum einen Werkstücke an das Conveyor Belt Module 2 um die Sortierung fortsetzen zu können und zum anderen an den Roboterarm am Ende

von Conveyor Belt Module 2 nach erfolgreicher Sortierung. Außerdem gibt es einen Informationsaustausch zwischen den beiden CBMs um evtl. Zustandsänderungen mitteilen zu können. Intern erkennt das System selbst Fehler, meldet diese dem Bediener und zeigt Ihm den aktuellen Zustand an.

3.1.5 Fehlerbehandlung

In diesem Kapitel beschreiben wir den Umgang mit Fehlern und deren Kategorisierung.

Fehlerarten: Sonderform, Werkstück verschwindet, Werkstück taucht auf, beide Rutschen voll & Werkstück soll aussortiert werden

Fehler wird erkannt: Alle Aktoren des CBM aus, Rote Lampe blinkt, Fehler anstehend unquittiert

Quittierungsprozess: Fehler anstehend unquittiert->Bediener quittiert Fehler über Reset Taste->Fehler anstehend quittiert (Lampe leuchtet durchgehend)->Bediener behebt Fehler->Bediener drückt auf Start Taste->Ruhezustand und Ampel geht aus

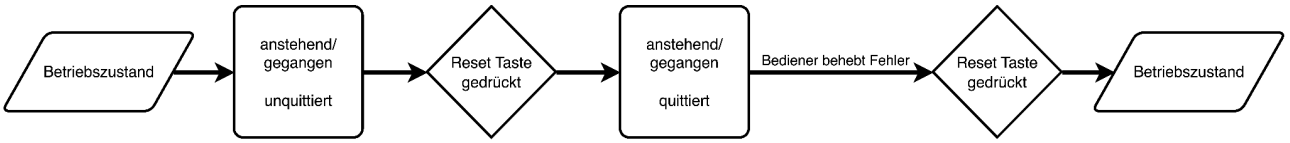


Abbildung 3.1.5.1: Ablaufdiagramm eines Quittierungsprozesses

3.1.6 Use Cases / User Stories

Hier werden zentrale Abläufe als User Stories definiert. Sie beschreiben das geforderte Systemverhalten und dienen als konsistente Grundlage für spätere Abnahmetests.

Werkstückerkennung

Überschrift	**EP_1 Werkstücke eindeutig erkennen, vermessen und typisieren**
Inhalt	Als Bediener möchte ich, dass jedes Werkstück beim Auflegen erkannt, mit einer eindeutigen ID versehen und anhand seiner Merkmale (Höhe, Metall, Bohrung) einem Typ zugeordnet wird, damit jedes Werkstück über seinen gesamten Weg verfolgt und korrekt sortiert werden kann.
Akzeptanzkriterien	**AK1:** Jedes Werkstück am Bandanfang 1 erhält eine eindeutige ID **AK2:** Höhe, metallisch ja/nein und Bohrungsgröße werden erfasst und unter der ID gespeichert **AK3:** Aus den Merkmalen wird der Typ bestimmt, unbekannte und defekte Werkstücke werden als Typ falsch markiert, Sonderform Werkstücke werden nach US_28 behandelt **AK4:** Werkstücke ohne ID werden erkannt und gemeldet **AK5:** Der Typ wird auf CBM 2 erneut bestimmt und mit den Daten von CBM 1 abgeglichen
User Stories	US_1 bis US_8

Überschrift	**US_1 Werkstück am Bandanfang erkennen und ID vergeben**
Inhalt	Als Bediener möchte ich Werkstücke am Anfang des CBM1 legen können und das System soll dieses erkennen und mit einer eindeutigen ID versehen, damit jedes Werkstück über seinen gesamten Weg eindeutig verfolgt werden kann.
Vorbedingungen	Die Anlage ist im Betriebszustand kein Werkstück vorher am Bandanfang
Akzeptanzkriterien	**AK1:** Ein aufgelegtes Werkstück wird erkannt und mit einer eindeutigen ID versehen
Quelle	25, 26, 39

Überschrift	**US_2 Höhe Messung eines Werkstückes einlesen und speichern**
Inhalt	Als Bediener möchte ich, dass die Höhe jedes Werkstücks beim Durchlauf der Höhenmessung automatisch erfasst und der ID zugeordnet wird, damit der Werkstück-Typ bestimmt werden kann.
Vorbedingungen	Werkstück wurde bereits eine ID zugewiesen Conveyor Belt läuft langsam Es befindet sich nur ein Werkstück unter dem Höhensensor
Akzeptanzkriterien	**AK1:** Bei einer unplausiblen Messung wird das Werkstück als defekt angesehen
Quelle	3, 31, 61 bis 63

Überschrift	**US_3 Bestimmen ob ein Werkstück metallisch ist**
Inhalt	Als Bediener möchte ich, dass das Conveyor Belt Module beim Durchlauf am Metallsensor erkennt, ob ein Werkstück Metall enthält, für die spätere Sortierreihenfolge.
Vorbedingungen	Werkstück wurde bereits eine ID zugewiesen Es befindet sich nur ein Werkstück unter dem Metallsensor
Akzeptanzkriterien	**AK1:** enthält Metall oder enthält kein Metall wird unter der ID gespeichert
Quelle	4, 8, 13

Überschrift	US_4 Bohrungsgröße des Werkstücks bestimmen
Inhalt	Als Bediener möchte ich, dass das System anhand des erfassten Höhenprofils die Bohrungsgröße des Werkstücks bestimmt (keine, klein, mittel, groß) und unter der ID speichert, damit die Werkstücke in der geforderten Reihenfolge sortiert werden können.
Vorbedingungen	Das Höhenprofil wurde korrekt eingelesen und unter seiner ID gespeichert
Akzeptanzkriterien	**AK1:** Werkstücke mit unterschiedlichen Bohrungen werden korrekt unterschieden
Quelle	4 bis 7

Überschrift	US_5 Werkstück Typ bestimmen und zur ID speichern
Inhalt	Als Bediener möchte ich, dass das System aus den erfassten Merkmalen eines Werkstücks (Höhe, Metall, Bohrung) dessen Typ klassifiziert und zusammen mit der ID speichert, damit die Reihenfolge und Aussortierung korrekt durchgeführt werden kann.
Vorbedingungen	Werkstück wurde bereits eine ID, Bohrungsgröße und Metall/nichtmetall zugewiesen
Akzeptanzkriterien	**AK1:** Passt die Kombination aus Bohrung und Metall nicht zu den bekannten Typen der Reihenfolge, wird das Werkstück als “falsches Werkstück” markiert **AK2:** ID und Typ sind spätestens am Ende von Band 2 für jedes Werkstück verfügbar
Quelle	36 bis 38, 40

Überschrift	US_6 Außerhalb des Bandanfangs eingelegte Werkstücke korrekt erkennen
Inhalt	Als Bediener möchte ich, dass ein Werkstück, das außerhalb des Bandanfangs auf ein Conveyor Belt Module gelangt, erkannt und gemeldet wird, damit keine unbekannten Werkstücke ungeprüft weiterlaufen.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wenn ein Sensor außerhalb des Bandanfangs ein Werkstück meldet, dem keine ID zugeordnet ist, stoppt das entsprechende Band unverzüglich **AK2:** Eine Fehlermeldung wird in Klartext auf der entsprechenden Konsole ausgegeben **AK3:** Die Ampel zeigt den Fehler als anstehend unquittiert an **AK4:** Nach entfernen des Werkstücks und festgelegtem Quittierungsprozess läuft die Anlage weiter
Quelle	70

Überschrift	US_7 Unbekannte, Defekte und Sonderform Werkstücke identifizieren
Inhalt	Als Bediener möchte ich, dass Werkstücke, die nicht zur vorgegebenen Reihenfolge gehören, erkannt und entsprechend markiert werden, damit sie nicht in die Reihenfolge gelangen und die nachfolgende Maschine nicht beschädigen.
Vorbedingungen	Werkstück wurde bereits eine ID, Bohrungsgröße und Metall/nichtmetall zugewiesen
Akzeptanzkriterien	**AK1:** Alle Unbekannten und Defekten Werkstücke werden sicher als Typ “falsches Werkstück" erkannt, ohne dass korrekte Werkstücke fälschlich markiert werden. Sonderform Werkstücke werden nach US_28 behandelt. **AK2:** Passt die Kombination aus Bohrung und Metall nicht zu den bekannten Typen, wird das Werkstück als Typ falsches Werkstück markiert
Quelle	10, 11, 21

Überschrift	US_8 Typ des Werkstückes auf CBM 2 verifizieren
Inhalt	Als Bediener möchte ich, dass der Typ eines Werkstückes auf Conveyor Belt Module 2 erneut bestimmt und mit den Daten von Conveyor Belt Module 1 verglichen wird, damit Messfehler oder Vertauschungen erkannt werden.
Vorbedingungen	ID und Typ des Werkstückes wurde korrekt identifiziert (CBM 1) ID und Typ wurde korrekt an CBM 2 übertragen
Akzeptanzkriterien	**AK1:** Stimmen sie überein, läuft das Werkstück weiter **AK2:** Typ oder Höhe weichen ab, wird es aussortiert, und der Grund wird beim Verlassen des Bandes auf der Konsole ausgegeben
Quelle	22

Sortierlogik und Reihenfolge

Überschrift	**EP_2 Werkstücke in der geforderten Reihenfolge ausgeben**
Inhalt	Als Betreiber des Systems möchte ich, dass nur Werkstücke in der vorgegebenen Reihenfolge das System verlassen und alle anderen aussortiert werden, damit die nachfolgende Maschine nicht beschädigt wird.
Akzeptanzkriterien	**AK1:** Die Werkstücke liegen spätestens nach der Rutsche von CBM 2 in der Reihenfolge nicht-metallisch/große Bohrung → metallisch/ohne Bohrung → nicht-metallisch/mittlere Bohrung → metallisch/kleine Bohrung vor **AK2:** Falsche Werkstücke werden bevorzugt auf CBM 1, sonst auf CBM 2 aussortiert **AK3:** Beim Verlassen des Bandes werden ID, Typ und Grund auf der Konsole ausgegeben **AK4:** Das nächste erwartete Werkstück wird auf Konsole 2 angezeigt
User Stories	US_9 bis US_12

Überschrift	**US_9 Reihenfolge der Werkstücke auf CBM2 sicherstellen**
Inhalt	Als Betreiber des Systems möchte ich, dass die Werkstücke spätestens nach der Rutsche von CBM2 in der korrekten Reihenfolge vorliegen, da sonst die nachfolgende Maschine beschädigt werden kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Am Ende von CBM2 kommen die Werkstücke in dieser Reihenfolge an: nicht-metallisches Werkstück mit großer Bohrung → metallisches Werkstück ohne Bohrung → nicht-metallisches Werkstück mit mittlerer Bohrung → metallisches Werkstück mit kleiner Bohrung. **AK2:** Passt ein Werkstück nicht zur erwarteten Position, wird es auf CBM 1 oder CBM 2 aussortiert **AK3:** Verhalten bei voller Rutsche siehe US_27
Quelle	1, 16, 17

Überschrift	**US_10 Reihenfolge der Werkstücke auf CBM 1 sicherstellen**
Inhalt	Als Betreiber des Systems möchte ich, dass Werkstücke, die nicht zur erwarteten Position der Reihenfolge passen, möglichst schon auf Conveyor Belt Module 1 aussortiert werden, damit Conveyor Belt Module 2 entlastet wird und der Durchsatz hoch bleibt.
Vorbedingungen	Rutsche von CBM 1 nicht voll
Akzeptanzkriterien	**AK1:** Reihenfolge: nicht-metallisches Werkstück mit großer Bohrung → metallisches Werkstück ohne Bohrung → nicht-metallisches Werkstück mit mittlerer Bohrung → metallisches Werkstück mit kleiner Bohrung **AK2:** Wenn die Rutsche von CBM 1 voll ist, wird die Aussortierung an CBM 2 übergeben (Siehe US_9)
Quelle	19, 20

Überschrift	US_11 Werkstücke die nicht der Reihenfolge entsprechen aussortieren
Inhalt	Als Betreiber des Systems möchte ich, dass Werkstücke, die nicht der Reihenfolge entsprechen, bevorzugt auf Conveyor Belt Module 1, sonst auf Conveyor Belt Module 2 aussortiert werden, damit die nachfolgende Maschine nicht beschädigt wird.
Vorbedingungen	min. eine Rutsche der beiden CBM ist nicht voll Werkstücke die nicht der Reihenfolge entsprechen, wurden korrekt identifiziert
Akzeptanzkriterien	**AK1:** Wenn die Rutsche von CBM 1 nicht voll ist, wird diese zuerst für die Aussortierung verwendet, ansonsten wird die Rutsche von CBM 2 verwendet, sofern diese nicht auch voll ist. **AK2:** Beim Verlassen des Bandes werden ID, Typ und Grund auf der entsprechenden Konsole ausgegeben
Quelle	19, 20 bis 24

Überschrift	US_12 Nächstes erwartetes Werkstück auf Konsole 2 ausgeben
Inhalt	Als Bediener möchte ich auf der Konsole von Conveyor Belt Module 2 sehen, welches Werkstück als Nächstes am Conveyor Beltende erwartet wird, damit ich den Ablauf vom System nachvollziehen kann.
Vorbedingungen	Das System ist im Betriebszustand
Akzeptanzkriterien	**AK1:** Die Anzeige wird aktualisiert sobald ein Werkstück das Bandende von CBM 2 verlassen hat **AK2:** Im Produktivbetrieb werden keine Entwickler-Ausgaben angezeigt (alles im Klartext)
Quelle	46

Materialfluss auf Conveyor Belt 1

Überschrift	US_13 Auf CBM 1 können immer Werkstücke aufgelegt werden
Inhalt	Als Bediener möchte ich jederzeit ein Werkstück auf CBM 1 legen können, solange der Conveyor Belt Anfang frei ist, damit das System kontinuierlich befüllen kann, ohne auf andere Werkstücke warten zu müssen.
Vorbedingungen	Conveyor Beltanfang von CBM 1 ist frei und das Conveyor Belt Module ist im Betriebszustand
Akzeptanzkriterien	**AK1:** Es können mehrere Werkstücke gleichzeitig auf CBM 1 sein, wobei alle ihre eigene ID besitzen **AK2:** Das Auflegen im laufenden Betrieb stört die Sortierung des restlichen Conveyor Belt Modules nicht
Quelle	27, 28

Materialfluss Allgemein

Überschrift	**EP_3 Zuverlässiger und wirtschaftlicher Transport auf beiden CBMs**
Inhalt	Als Betreiber des Systems möchte ich, dass die Werkstücke auf beiden CBMs zuverlässig, sicher und mit hohem Durchsatz transportiert werden, damit die Messwerte korrekt sind, keine Werkstücke herunterfallen und das System wirtschaftlich bleibt.
Akzeptanzkriterien	**AK1:** Die Conveyor Belter fahren nur während der Höhenmessung langsam **AK2:** Ein leeres Conveyor Belt stoppt selbstständig, bei neuem Werkstück läuft es wieder an **AK3:** Es fällt zu keinem Zeitpunkt ein Werkstück vom Band, auch nicht bei Fehlern oder E-Stop **AK4:** Der Durchsatz wird maximiert, Aussortierung auf CBM 1 hat Vorrang
User Stories	US_14 bis US_17

Überschrift	**US_14 Werkstücke fahren langsam durch die Höhenmessung**
Inhalt	Als Betreiber des Systems möchte ich, dass Werkstücke auf beiden CBMs langsam durch die Höhenmessung transportiert werden, damit die Höhenmessung zuverlässige Werte liefert.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** ein Werkstück befindet sich an der Höhenmessung und das Conveyor Belt wechselt in die langsame Geschwindigkeit **AK2:** Nach Verlassen des Werkstücks aus der Höhenmessung wechselt das Conveyor Belt wieder in die normale Geschwindigkeit
Quelle	31

Überschrift	**US_15 Band Stoppt, wenn kein Werkstück drauf ist**
Inhalt	Als Betreiber des Systems möchte ich, dass ein Conveyor Belt stoppt, sobald sich kein Werkstück darauf befindet, damit Energie gespart und Verschleiß vermieden wird.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Befinden sich keine Werkstücke auf einem Conveyor Belt Module, so stoppt der Motor des Conveyor Beltes **AK2:** Wird ein neues Werkstück aufgelegt bzw. übergeben, läuft der Motor des Conveyor Beltes wieder an **AK3:** Ein Conveyor Belt ist leer, wenn kein Werkstück mehr von einem der Sensoren erfasst wird und die Verfolgung der Werkstücke kein Werkstück mehr beinhaltet **AK4:** Jedes Conveyor Belt Module entscheidet hierbei autark
Quelle	43

Überschrift	**US_16 Sicherstellen das keine Werkstücke runterfallen können**
Inhalt	Als Betreiber des Systems möchte ich, dass zu keinem Zeitpunkt ein Werkstück von einem Conveyor Belt fällt, damit die Werkstücke nicht beschädigt und die Verfolgung konstant bleibt.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wenn ein Werkstück am Conveyor Beltende von CBM 2 kommt, stoppt das Conveyor Belt, bis das Werkstück von dem Conveyor Belt entnommen wurde **AK2:** Bei Fehlern oder E-Stop bleiben die Conveyor Belter sofort stehen, damit keine Werkstücke runterfallen können
Quelle	32

Überschrift	US_17 Maximalen Durchsatz von Werkstücken sicherstellen
Inhalt	Als Betreiber des Systems möchte ich, dass das System möglichst viele Werkstücke pro Zeit sortiert, damit das System wirtschaftlich bleibt.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Conveyor Belter laufen nur für ein kurzes vordefiniertes Zeitfenster bei der Höhenmessung langsam (verweis auf US_14 AK1 & AK2) **AK2:** CBM2 sollte 2 Werkstücke transportieren (maximale Kapazität) **AK3:** Aussortierung auf CBM 1 hat Vorrang um CBM 2 zu *entlasten*
Quelle	44

Materialfluss auf Band 2

Überschrift	EP_4 Geordnete Übergabe und Förderung auf CBM 2
Inhalt	Als Betreiber des Systems möchte ich, dass Werkstücke vereinzelt an CBM 2 übergeben und dort mit definiertem Abstand gefördert werden, damit Prüfung, Aussortierung und Entnahme zuverlässig funktionieren.
Akzeptanzkriterien	**AK1:** Werkstücke werden einzeln von CBM 1 an CBM 2 übergeben **AK2:** Es befinden sich maximal zwei Werkstücke mit mindestens 25 cm Abstand auf CBM 2 **AK3:** CBM 2 stoppt, solange ein Werkstück am Bandende liegt **AK4:** ID und Typ werden von CBM 1 an CBM 2 übertragen, bei Ausfall der Qnet-Verbindung reagiert das System definiert
User Stories	US_18 bis US_22

Überschrift	US_18 Werkstücke vereinzelt übergeben an CBM2
Inhalt	Als Betreiber des Systems möchte ich, dass Werkstücke einzeln von Conveyor Belt Module 1 an Conveyor Belt Module 2 übergeben werden, damit sie dort *sicher geprüft* und in der richtigen Reihenfolge ausgegeben werden können.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Es wird immer nur 1 Werkstück einzeln von CBM1 an CBM2 übergeben **AK2:** CBM2 darf nicht mehr als 2 Werkstücke mit einem Abstand von min. 25 cm fördern **AK3:** Ist CBM2 voll, so wartet das Werkstück auf CBM1
Quelle	29

Überschrift	US_19 Mindestabstand von 25 cm einhalten
Inhalt	Als Betreiber des Systems möchte ich, dass der Abstand zwischen Werkstücken auf CBM2 mindestens 25 cm beträgt, damit Prüfung, Aussortierung und Entnahme einzelner Werkstücke zuverlässig funktionieren.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Befinden sich 2 Werkstücke auf CBM2, so beträgt deren Abstand immer min. 25 cm **AK2:** Es kann erst ein zweites Werkstück an CBM2 übergeben werden, wenn der Abstand von 25 cm sichergestellt werden kann
Quelle	30, Zusammenhang US_18

Überschrift	US_20 Maximal 2 Werkstücke auf CBM2
Inhalt	Als Betreiber des Systems möchte ich, dass sich höchstens zwei Werkstücke gleichzeitig auf CBM 2 befinden, damit die Prüfung und die Entnahme am Conveyor Beltende kontrollierbar bleiben.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** CBM2 darf nicht mehr als 2 Werkstücke mit einem Abstand von min. 25 cm transportieren **AK2:** Die Zählung stimmt auch nach Aussortieren, Entnahme und Fehlern (zum Beispiel verschwundenes Werkstück)
Quelle	29, Zusammenhang US_18

Überschrift	US_21 CBM2 stoppt solange ein Werkstück am Conveyor Beltende liegt
Inhalt	Als Betreiber des Systems möchte ich, dass CBM2 stoppt, sobald ein Werkstück das Conveyor Beltende erreicht, und erst weiterläuft, wenn es entnommen wurde, damit der Roboter es sicher greifen kann und das Werkstück nicht herunterfällt.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Der Motor von CBM2 stoppt, wenn ein Werkstück die Lichtschranke am Conveyor Beltende erreicht **AK2:** Die entsprechenden Daten (ID, Typ, Grund) zu dem Werkstück werden auf der Konsole angezeigt, sobald es vom Conveyor Beltende entnommen wurde
Quelle	18, 35

Überschrift	US_22 Werkstück Daten und Status werden an CBM2 übermittelt
Inhalt	Als Betreiber des Systems möchte ich, dass ID und Typ jedes Werkstücks von CBM1 an CBM2 übertragen werden, damit CBM2 das Werkstück erneut prüfen und die Daten am Bandende ausgeben kann.
Vorbedingungen	Das Werkstück hat auf CBM1 ID und Typ erhalten
Akzeptanzkriterien	**AK1:** Bei Ausfall der Qnet Verbindung reagiert das System definiert **AK2:** ID und Typ sind auf CBM2 bekannt, wenn das Werkstück dort ankommt
Quelle	22, 36 bis 40

Werkstücke Aussortieren

Überschrift	EP_5 Falsche und nicht sortierbare Werkstücke sicher aussortieren
Inhalt	Als Betreiber des Systems möchte ich, dass nicht passende Werkstücke zuverlässig über Weiche oder Auswerfer aussortiert werden und der Bediener über volle Rutschen oder nicht sortierbare Werkstücke informiert wird, damit die Reihenfolge eingehalten und die Hardware geschützt wird.
Akzeptanzkriterien	**AK1:** Weiche und Auswerfer sind in jeder Kombination an beiden CBMs einsetzbar **AK2:** Die Weiche wird nur so lange wie nötig bestromt und überhitzt nie **AK3:** Der Auswerfer drückt nur zu aussortierende Werkstücke in die Rutsche und fährt danach wieder ein **AK4:** Eine volle Rutsche wird am betroffenen CBM signalisiert **AK5:** Sind beide Rutschen voll und ein Werkstück muss aussortiert werden, stoppt das System kontrolliert **AK6:** Nicht sortierbare Werkstücke werden dem Bediener zur Entnahme gemeldet
User Stories	US_23 bis US_28

Überschrift	US_23 Überhitzen der Weiche vermeiden
Inhalt	Als Betreiber des Systems möchte ich, dass die Weiche nur so lange bestromt wird bis das Werkstück *sicher* die Weiche passiert hat, damit die Magnetspule nicht überhitzt und die Weiche nicht beschädigt wird.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Die Weiche wird zum Öffnen bestromt, dann wird der Strom abgeschaltet, sobald das Werkstück durch ist, spätestens nach einer *vorher* festgelegten Maximaldauer. **AK2:** Nach dem Abschalten ist die Weiche wieder geschlossen **AK3:** Die Weiche wird nie dauerhaft oder über Minuten bestromt, auch nicht bei Fehlern, E-Stop oder blockiertem Werkstück. **AK4:** Eventuell Warning bei Stop Taste
Quelle	53 bis 55

Überschrift	US_24 Auswerfer ansteuern wenn Werkstück vor der Rutsche liegt
Inhalt	Als Betreiber des Systems möchte ich, dass der Auswerfer ein auszusortierendes Werkstück, wenn es sich vor dem Auswerfer befindet, in die Rutsche drückt, damit es aussortiert wird und kein anderes Werkstück beeinträchtigt wird.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wenn sich ein Werkstück vor der Rutsche befindet und aussortiert werden soll, wird dieses durch die Ansteuerung des Auswerfers in die Rutsche gedrückt **AK2:** Wenn ein Werkstück die Rutsche ohne aussortiert zu werden passieren möchte, bleibt der Auswerfer stromlos **AK3:** Nach dem Auswerfen fährt der Auswerfer wieder ein, damit andere nachfolgende Werkstücke passieren können
Quelle	50, 56 bis 58

Überschrift	US_25 Beide Aussortier Varianten können verwendet werden
Inhalt	Als Betreiber des Systems möchte ich, dass mit Weiche und Auswerfer in jeder Kombination der beiden CBMs gearbeitet werden kann, damit ich das System ohne Softwareänderung mit beiden Varianten betreiben kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Das System funktioniert mit Weiche und Weiche, Weiche und Auswerfer, Auswerfer und Weiche, sowie Auswerfer und Auswerfer. **AK2:** Die Varianten werden bei der Kalibrierung im Service-Mode erkannt
Quelle	59, 60

Überschrift	US_27 Bei 2 vollen Rutschen und einem falschen Werkstück wird gestoppt
Inhalt	Als Betreiber des Systems möchte ich, dass das System bei einem auszusortierenden Werkstück ohne freie Rutsche kontrolliert stoppt, damit die Reihenfolge eingehalten werden kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Beide Rutschen voll und kein Werkstück muss aussortiert werden → Conveyor Belt läuft weiter **AK2:** Beide Rutschen voll und ein Werkstück muss aussortiert werden → Conveyor Belt stoppt unverzüglich **AK3:** Die Ampel zeigt den Fehler als anstehend unquittiert an **AK4:** Nach Leeren der Rutschen läuft das System nach Quittierungsprozess weiter
Quelle	34, 71

Überschrift	US_26 Eine volle Rutsche wird am entsprechenden CBM signalisiert
Inhalt	Als Bediener möchte ich an jedem Conveyor Belt Module erkennen, ob dessen Rutsche voll ist, damit ich sie rechtzeitig leeren kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Es wird nur an dem betroffenen CBM signalisiert **AK2:** Wird die Rutsche geleert, endet die Signalisierung **AK3:** Erkennung durch Hinweise auf der entsprechenden Konsole
Quelle	33

Überschrift	US_28 Nicht sortierbare Werkstücke zur Entnahme durch Bediener melden
Inhalt	Als Bediener möchte ich informiert werden, wenn ein Werkstück nicht von dem System sortiert werden kann, damit ich es entferne und das System weiterlaufen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Die entsprechende Konsole zeigt im Klartext, welches Werkstück (ID, Typ, Grund) wo auf dem Conveyor Belt entfernt werden muss. **AK2:** Nach Entfernung des nicht sortierbaren Werkstücks und dem Quittierungsprozess läuft das Conveyor Belt Module normal weiter.
Quelle	45

Fehlerbehandlung

Überschrift	EP_6 Fehler erkennen, lokalisieren und definiert behandeln
Inhalt	Als Bediener möchte ich, dass Fehler erkannt, verständlich mit Ort gemeldet und nach einem festen Ablauf quittiert und behoben werden können, damit das System sicher weiterläuft und die Werkstückverfolgung konsistent bleibt.
Akzeptanzkriterien	**AK1:** Verschwundene und nachträglich aufgelegte Werkstücke werden erkannt und gemeldet **AK2:** Jeder Fehler hat eine eindeutige Klartextmeldung mit Ort und, falls vorhanden, Werkstück (ID, Typ) **AK3:** Ein Fehler stoppt nur das betroffene CBM, außer er betrifft beide **AK4:** Fehler durchlaufen die Zustände „anstehend unquittiert“ → „anstehend quittiert“ → „kein Fehler“, bzw. „gegangen unquittiert“ bei Selbstkorrektur
User Stories	US_29 bis US_33

Überschrift	US_29 Verschwundenes Werkstück erkennen
Inhalt	Als Bediener möchte ich, dass ein Werkstück, das von dem System verschwindet, erkannt und gemeldet wird, damit keine falsche Reihenfolge entsteht und die Werkstückverfolgung konsistent bleibt.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wird ein Werkstück zwischen zwei Lichtschranken vom Conveyor Belt genommen, stoppt das CBM und meldet den Fehler. **AK2:** Nach Behebung des Fehlers und Quittierungsprozess läuft das Conveyor Belt Module weiter **AK3:** entsprechendes Werkstück wird aus der Verfolgungsliste entfernt **AK4:** Auf der entsprechenden Konsole wird angezeigt, welches Werkstück (ID, Typ) wo auf dem Conveyor Belt fehlt **AK5:** Die Ampel zeigt den Fehler als anstehend unquittiert an
Quelle	67 bis 69

Überschrift	US_30 Hinzufügen von Werkstücken außerhalb des Conveyor Beltanfangs
Inhalt	Als Bediener möchte ich, dass ein Werkstück, das außerhalb des Conveyor Beltanfangs auf ein Conveyor Belt gelangt, *erkannt und gemeldet* wird, damit keine unbekannten Werkstücke ungeprüft weiterlaufen.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Auf der entsprechenden Konsole wird angezeigt, wo auf dem Conveyor Belt das Werkstück ohne ID erkannt wurde **AK2:** Das Auflegen am Conveyor Beltanfang von CBM1 löst keinen Fehler aus, wenn die Leuchte Q1 leuchtet **AK3:** Die Ampel zeigt den Fehler als anstehend unquittiert an
Quelle	70

Überschrift	US_31 Fehler mit Lokalisierung in Klartext auf der Konsole
Inhalt	Als Bediener möchte ich bei einem Fehler eine verständliche Meldung mit Ort der Fehlerstelle erhalten, damit ich den Fehler schnell finden und beheben kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Die Meldung zeigt Werkstück (ID, Typ) wenn vorhanden **AK2:** Die Meldung nennt den Bereich, z. B. zwischen Einlauf und Höhenmessung **AK3:** Jeder Fehler hat eine eigene eindeutige Meldung im Klartext **AK4:** Die Meldung verschwindet nach Beheben des Fehlers und anschließendem Quittierungsprozess
Quelle	66

Überschrift	US_32 Fehlerroutine nur am entsprechenden CBM behandeln
Inhalt	Als Betreiber des Systems möchte ich, dass ein Fehler, der nur ein CBM betrifft, nur dieses Conveyor Belt stoppt und das andere CBM weiterlaufen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Bei einem Fehler stoppt nur das betroffene Conveyor Belt **AK2:** Die Fehlermeldung wird nur auf der entsprechenden Konsole ausgegeben **AK3:** Sollte der Fehler beide CBMs betreffen, gehen beide in den Fehlerzustand
Quelle	47, 48

Überschrift	US_33 Fehlerroutine nach Vorgaben durchführen
Inhalt	Als Bediener möchte ich einen Fehler quittieren und beheben können, sobald dieser ansteht, um einen sicheren Ablauf des Systems gewährleisten zu können.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Der Fehler durchläuft die Zustände: kein Fehler → anstehend unquittiert → anstehend quittiert → kein Fehler **AK2:** neue Fehler sind „anstehend unquittiert“ und werden über die Reset-Taste auf „anstehend quittiert“ gesetzt, um dann behoben zu werden **AK3:** Fehler, die sich selbst korrigieren, haben den Status „gegangen unquittiert“, bis diese quittiert werden
Quelle	72, 73, 94 bis 103

Bedienung

Überschrift	EP_7 System über Tasten bedienen
Inhalt	Als Bediener möchte ich das System über Start-, Stop- und Reset-Taste einfach und eindeutig steuern können, damit ich es in den gewünschten Zustand versetzen, kalibrieren und Fehler quittieren kann.
Akzeptanzkriterien	**AK1:** Kurzes Drücken der Start-Taste (<1 s) versetzt das CBM in den Betriebszustand **AK2:** Langes Drücken (>3 s) startet den Service-Mode **AK3:** Die Stop-Taste versetzt das CBM in den Ruhezustand, Daten und Reihenfolge bleiben erhalten **AK4:** Die Reset-Taste quittiert Fehler gemäß Fehlerzustand
User Stories	US_34 bis US_37

Überschrift	US_34 System per Start-Taste in den Betriebszustand bringen
Inhalt	Als Bediener möchte ich das System per Start-Taste in den Betriebszustand versetzen, damit es Werkstücke am Conveyor Beltanfang des CBMs entgegen nehmen kann.
Vorbedingungen	CBM befindet sich im Ruhezustand und es liegen keine Fehler vor
Akzeptanzkriterien	**AK1:** wenn die Start-Taste kurz gedrückt wird (<1 Sekunde), wechselt das CBM in den Betriebszustand **AK2:** im Betriebszustand leuchtet die grüne Lampe der Ampel des CBM dauerhaft
Quelle	75, 76

Überschrift	US_35 Service-Mode per langem Druck starten
Inhalt	Als Bediener möchte ich durch langes Drücken der Start-Taste eines CBMs in den Service-Mode wechseln, damit ich Kalibrierungen durchführen kann.
Vorbedingungen	CBM ist im Ruhezustand
Akzeptanzkriterien	**AK1:** wenn die Start-Taste lang gedrückt wird (>3 Sekunden), wechselt das CBM in den Service-Mode und die grüne Lampe der Ampel blinkt **AK2:** der Service-Mode wird über die Stop-Taste verlassen (in den Ruhezustand) **AK3:** Kalibrierungen und Reihenfolge werden zurückgesetzt
Quelle	77

Überschrift	US_36 CBM per Stop-Taste in den Ruhezustand bringen
Inhalt	Als Bediener möchte ich das CBM per Stop-Taste in den Ruhezustand versetzen, damit ich es geordnet anhalten kann.
Vorbedingungen	CBM ist im Betriebszustand oder Service-Mode und es liegen keine Fehler oder Warnungen vor
Akzeptanzkriterien	**AK1:** über ein Drücken der Stop-Taste wechselt das CBM sofort in den Ruhezustand **AK2:** im Ruhezustand ist das Conveyor Belt angehalten und die Lampen der Ampel aus **AK3:** Daten und Reihenfolgezähler bleiben erhalten, damit das CBM nach erneutem Wechsel in den Betriebszustand konsistent weiterläuft
Quelle	78, 79

Überschrift	US_37 Fehler per Reset-Taste quittieren
Inhalt	Als Bediener möchte ich einen Fehler mit der Reset-Taste quittieren, damit das CBM weiß, dass ich den Fehler gesehen habe, und die Leuchte der Ampel entsprechend wechselt.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** wenn Fehler gegeben „anstehend unquittiert“, wechselt der Fehlerzustand durch Drücken der Reset-Taste in „anstehend quittiert“ und der Fehler kann behoben werden. **AK2:** wenn Fehler gegeben „gegangen unquittiert“, wechselt der Fehlerzustand durch Drücken der Reset-Taste in „kein Fehler“ **AK3:** Die Lampen der Ampel werden je nach Fehlerzustand angesteuert
Quelle	80

E-Stop

Überschrift	EP_8 System im Gefahrenfall sofort und sicher stilllegen
Inhalt	Als Bediener möchte ich das gesamte System per E-Stop sofort in einen sicheren Zustand versetzen und es erst nach bewusstem Entsperren wieder starten können, damit Personen und Anlage in Gefahrensituationen geschützt sind.
Akzeptanzkriterien	**AK1:** Der E-Stop wirkt aus jedem Zustand ohne Verzögerung, Motoren und Weichen/Auswerfer sind sofort stromlos **AK2:** Der E-Stop wirkt über Qnet auf beide CBMs **AK3:** Der Ursprung des E-Stops wird auf beiden Konsolen im Klartext angezeigt **AK4:** Der Neustart ist erst nach Lösen des E-Stops und Reset an beiden CBMs möglich **AK5:** Das E-Stop-Verhalten ist verständlich dokumentiert
User Stories	US_38 bis US_42

Überschrift	US_38 System per E-Stop sofort stilllegen
Inhalt	Als Bediener möchte ich das System mit dem E-Stop in einen sicheren Zustand versetzen, damit Personen und System in einer Gefahrensituation geschützt sind.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Aus jedem beliebigen Zustand kann der E-Stop durch Drücken eines Schalters ohne Verzögerung ausgelöst werden **AK2:** Die Motoren und Weichen/Auswerfer werden sofort stromlos geschaltet **AK3:** Solange der E-Stop aktiv ist, ist keine Bedienung des Systems möglich
Quelle	64, 65, 81

Überschrift	US_39 E-Stop wirkt auf beide CBMs
Inhalt	Als Bediener möchte ich, dass ein E-Stop an einem Bedienpanel das gesamte System in einen sicheren Zustand versetzt, somit auch das CBM steht, an dem ich nicht gedrückt habe.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Bei Drücken des E-Stop an CBM 1 leitet auch CBM 2 unverzüglich den E-Stop ein **AK2:** Der E-Stop wird sofort über Qnet an das andere CBM übertragen
Quelle	82

Überschrift	US_40 Ursprung des E-Stops signalisieren
Inhalt	Als Bediener möchte ich erkennen, an welchem Conveyor Belt der E-Stop ausgelöst wurde, damit ich den richtigen E-Stop Schalter lösen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Die Meldung wird im Klartext mit dem entsprechenden CBM auf beiden Konsolen angezeigt **AK2:** Die Anzeige bleibt so lange bestehen, bis an beiden CBMs der E-Stop gelöst und anschließend mit Reset-Taste bestätigt wurde **AK3:** Auch bei Drücken beider E-Stops werden diese auf den Konsolen korrekt ausgegeben
Quelle	83

Überschrift	US_41 Nach E-Stop erst nach Reset an beiden CBMs weiterlaufen
Inhalt	Als Bediener möchte ich, dass das System nach einem E-Stop erst weiterläuft, wenn ich den Schalter gelöst und an beiden CBMs Reset gedrückt habe, damit es nicht unerwartet anläuft.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** System bleibt nach Lösen der E-Stops weiter stehen, bis beide Reset-Tasten einmal betätigt wurden **AK2:** Ein Drücken der Reset-Taste bei noch betätigtem E-Stop hat keine Wirkung **AK3:** Ein Umgehen der Reset-Tasten nach Lösen des E-Stops ist ausgeschlossen
Quelle	84

Überschrift	US_42 E-Stop-Verhalten verständlich dokumentieren
Inhalt	Als Bediener und Auftraggeber möchte ich eine einfach zu folgende und verständliche Dokumentation des E-Stop-Verhaltens, damit ich das System sicher bedienen und das Verhalten nachvollziehen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** *Die Dokumentation enthält mögliche Szenarien* **AK2:** Sie enthält eine übersichtliche Darstellung des E-Stop-Verhaltens (evtl. Zustandsdiagramm)
Quelle	85, 86

Signalisierung und Ausgabe

Überschrift	EP_9 Systemzustand über Ampel und Konsole nachvollziehbar anzeigen
Inhalt	Als Bediener möchte ich den Zustand des Systems, Warnungen, Fehler und den Weg der Werkstücke jederzeit über Ampel und Konsole erkennen können, damit ich den Ablauf nachvollziehen und rechtzeitig eingreifen kann.
Akzeptanzkriterien	**AK1:** Grün: Betriebszustand dauerhaft, Service-Mode blinkend **AK2:** Gelb: Warnungen blinkend **AK3:** Rot: Fehlerzustände (unquittiert 1 Hz, quittiert Dauerlicht, gegangen unquittiert 0,5 Hz) **AK4:** Beim Verlassen eines CBM werden ID, Typ und Grund pro Werkstück genau einmal ausgegeben **AK5:** Im Produktivbetrieb erscheinen nur Betriebsmeldungen im Klartext, keine Entwicklerausgaben
User Stories	US_43 bis US_47

Überschrift	US_43 Ampel grün: Betriebszustand dauerhaft, Service-Mode blinkend
Inhalt	Als Bediener möchte ich an der grünen Lampe der Ampel erkennen, ob das CBM im Betriebszustand oder im Service-Mode ist, damit ich den Zustand auf einen Blick sehe.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Ist das CBM im Betriebszustand, leuchtet die grüne Lampe der Ampel dauerhaft **AK2:** Ist das CBM im Service-Mode, blinkt die grüne Lampe der Ampel **AK3:** Bei anderen Zuständen leuchtet die grüne Lampe der Ampel nicht **AK4:** Jedes CBM steuert seine Ampel entsprechend seinem Zustand
Quelle	89 bis 91

Überschrift	US_44 Ampel gelb: Warnungen blinkend
Inhalt	Als Bediener möchte ich an der gelben Lampe der Ampel erkennen, dass eine Warnung vorliegt, damit ich eingreifen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wenn eine Warnung an dem entsprechenden CBM vorliegt, blinkt die gelbe Lampe **AK2:** liegt keine Warnung mehr vor, geht die Lampe wieder aus **AK3:** Eine Warnung verhindert den Wechsel in den Ruhezustand per Stop-Taste
Quelle	92

Überschrift	**US_45 Ampel rot: Fehlerzustände mit schnellem und langsamem Blinken**
Inhalt	Als Bediener möchte ich an der roten Lampe der Ampel den Zustand eines Fehlers erkennen und informiert werden.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Wenn kein Fehler vorliegt, ist die rote Lampe der Ampel aus **AK2:** Anstehend unquittiert: schnelles Blinken mit 1 Hz **AK3:** Anstehend quittiert: Dauerlicht **AK4:** Gegangen unquittiert: langsames Blinken mit 0,5 Hz
Quelle	93, 105 bis 109

Überschrift	**US_46 Werkstückdaten mit Grund bei Verlassen des CBM anzeigen**
Inhalt	Als Bediener möchte ich auf der Konsole sehen, welches Werkstück das Conveyor Belt verlassen hat und warum, damit ich den Ablauf nachvollziehen und prüfen kann.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Verlässt ein Werkstück ein Conveyor Belt, erscheinen ID, Typ und Grund auf der entsprechenden Konsole des jeweiligen CBM **AK2:** Jedes Werkstück erzeugt genau eine Ausgabe pro CBM
Quelle	41, 42

Überschrift	**US_47 Im normalen Betrieb keine Entwickler Ausgaben**
Inhalt	Als Betreiber möchte ich, dass im Produktivbetrieb nur Betriebsmeldungen auf der Konsole erscheinen, damit sie übersichtlich bleiben.
Vorbedingungen	Keine
Akzeptanzkriterien	**AK1:** Keine Entwicklungsausgaben während des Produktivbetriebes **AK2:** Vordefinierte Betriebsmeldungen im Klartext
Quelle	49

3.2 Anlage: Analyse der technischen Gegebenheiten

Dieser Abschnitt gibt einen umfassenden Überblick über die physische Systemarchitektur der Anlage. Er beschreibt den modularen Aufbau aus zwei baugleichen CBMs, die Steuerung über die Steuerungsrechner sowie die netzwerkbasierte Kommunikation über das Qnet-Protokoll.

Das System besteht aus zwei baugleichen Conveyor Belt Modules (CBM1 und CBM2) des Festo-Transfersystems. Beide Module können je nach Anlagenkonfiguration mit einer Weiche oder einem Auswerfer zum Aussortieren der Werkstücke ausgestattet sein. Die Zuordnung der Aussortiermechanismen zu den Modulen ist dabei variabel. Jedes Conveyor Belt Module wird von einem eigenen BeagleBone Black (BBB) gesteuert. Die beiden Steuerungseinheiten sind über Ethernet miteinander verbunden und kommunizieren über das Qnet-Protokoll.//

3.2.1 Technischer Aufbau und Hardwarekomponenten

Hier wird der konkrete Weg eines Werkstücks in Transportrichtung von der Einlauflichtschranke über die Sensorik bis zum Auslauf oder der Weiche detailliert beschrieben. Die dazugehörigen Sensor- und Aktortabellen listen alle relevanten Komponenten inklusive ihrer funktionalen Bezeichner auf.

Aufbau eines Conveyor Belt Modules (in Transportrichtung):
Einlauf → Höhenmessung → Metallsensor / Weiche bzw. Auswerfer → Rutsche (Aussortierung) oder Auslauf (Weitergabe)

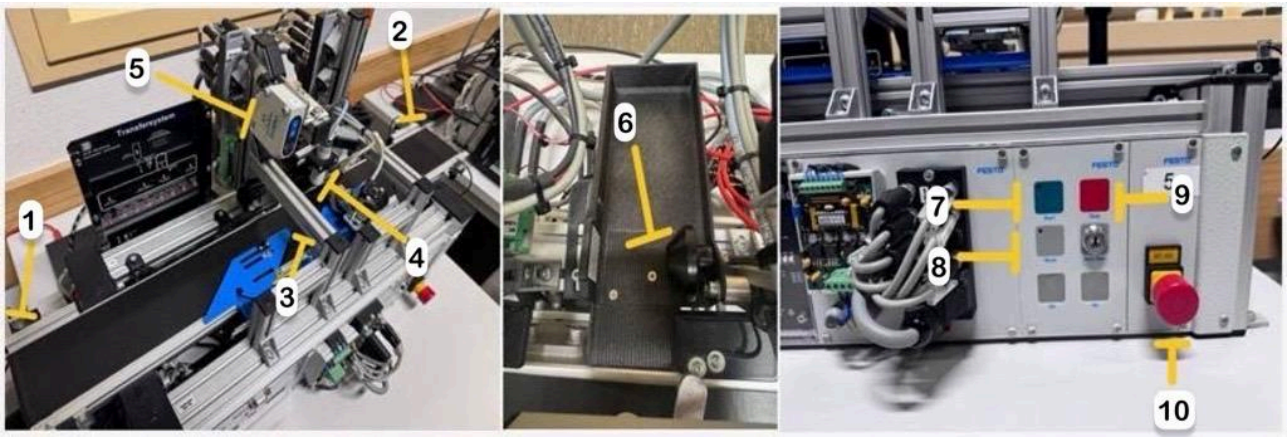


Abbildung 3.2.1.1: Übersicht der Conveyor Beltanlage mit den relevanten Sensoren

Tabelle 3.2.1.1: Sensorik

Nr.	Bezeichner	Komponente	Funktionalität
1	L_Anfang	Lichtschranke Einlauf	Erkennt ein neu aufgelegtes Werkstück
2	L_Ende	Lichtschranke Auslauf	Werkstück am Bandende
3	L_Aussortierer	Lichtschranke Weiche/ Auswerfer	Werkstück am Aussortiermechanismus
4	MD	Metalldetektor	Erkennt Metalleinsatz
5	H_Sensor	Höhenmesssensor	Bestimmung des Höhenprofils (Bohrung)

6	L_Rutsche	Lichtschranke Rutsche	Erkennt wenn die Rutsche voll ist
7	K_Start	Start-Knopf	Betrieb starten; lang drücken: Service-Mode
8	K_Reset	Reset-Knopf	Fehler quittieren
9	K_Stop	Stop-Knopf	In den Ruhezustand wechseln
10	E_Stop	Emergency-Stop-Knopf	Sofortiger Betriebsstopp aller Anlagen

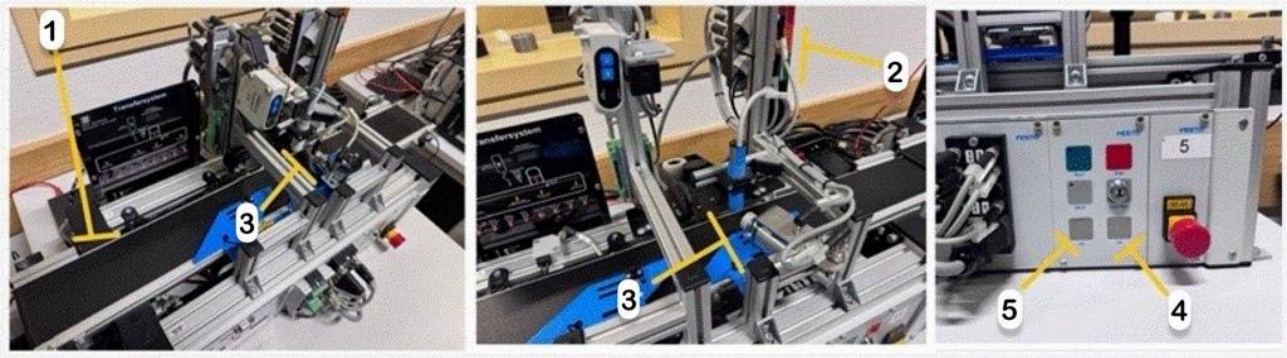


Abbildung 3.2.1.2: Übersicht der Conveyor Beltanlage mit den relevanten Aktoren

Tabelle 3.2.1.2: Aktorik

Nr.	Bezeichner	Komponente	Funktionalität
1	M_L	Motor Laufband	Bewegen des Conveyor Belts
2	AM_G	Ampel Grün	Leuchten und Signalisieren im Betriebszustand
2	AM_R	Ampel Rot	Leuchten und Signalisieren im Fehlerzustand
2	AM_Y	Ampel Gelb	Leuchten und Signalisieren im Warnzustand
3	W*(A)	Weiche /Auswerfer	Aussortieren von Werkstücken
4	LED_Q1	Signalleuchte Q1	Bedienhinweis

5	LED_Q2	Signalleuchte Q2	Bedienhinweis

3.2.2 Werkstücke

Dieser Abschnitt widmet sich den zu verarbeitenden Werkstücken und deren physikalischen Eigenschaften.

Die Sortieranlage verarbeitet zylindrische Werkstücke mit einem einheitlichen Außendurchmesser von 44 mm (physikalisch gemessen). Sie unterscheiden sich in Höhe, Farbe, Material und Aufbau. Neben flachen und hohen Varianten gibt es Werkstücke mit und ohne Bohrung. Bei einigen ist ein metallischer Einsatz in die Bohrung eingepresst, der vom Metallsensor der Anlage erkannt wird. Die Werkstücke sind in den Farben Rot, Schwarz, Weiß, Grau, Silber sowie semitransparent ausgeführt. Je nach Volumen, Material und Metalleinsatz variieren außerdem Gewicht und Gleiteigenschaften auf dem Conveyor Belt. Zusätzlich gibt es binär-codierte Werkstücke, deren Code über ein Rillenmuster auf der Oberseite abgebildet ist.

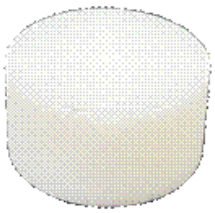


Abbildung 3.2.2.1: Werkstück flach, ohne Metalleinsatz, ohne Bohrung, weiß

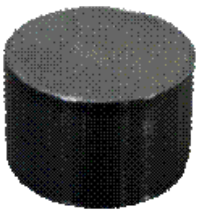


Abbildung 3.2.2.2: Werkstück hoch, ohne Metalleinsatz, ohne Bohrung, schwarz



Abbildung 3.2.2.3: Werkstück hoch, ohne Metalleinsatz, große Bohrung, silber



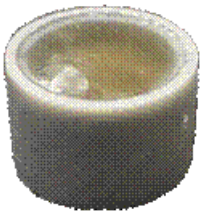
Abbildung 3.2.2.4: Werkstück hoch, mit Metalleinsatz, kleine Bohrung, mit Rille, rot



Abbildung 3.2.2.5: Werkstück hoch, mit Metalleinsatz, kleine Bohrung, weiß



Abbildung 3.2.2.6: Werkstück hoch, ohne Metalleinsatz, mittlere Bohrung, weiß

 *Abbildung 3.2.2.7: Werkstück hoch, ohne Metalleinsatz, ohne Bohrung, semitransparent*		

3.2.3 Steuergerät

Hier werden die Hardware- und Betriebssystem-Spezifikationen des Steuerungsrechners erläutert.

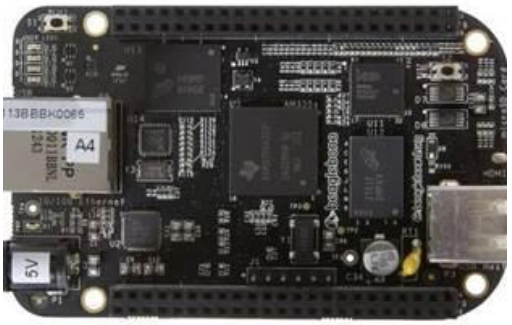


Abbildung 3.2.3.1: BeagleBone Black

jedes Conveyor Belt Module (CBM1, CBM2) wird von einem eigenen Steuerungsrechner vom Typ BeagleBone Black gesteuert. Kern des BBB ist das System-on-Chip TI AM335x mit einem ARM-Cortex-A8-Prozessor. Darauf läuft das QNX Betriebsystem, auf dem die Steuerungssoftware ausgeführt wird. Die beiden Steuerungsrechner sind über Ethernet gekoppelt und tauschen über das Qnet-Protokoll Informationen aus, zum Beispiel zur Werkstückübergabe und zum E-Stop.

Die digitalen Signale laufen über die General-Purpose-Input/Output-Module (GPIO) des AM335x. Jedes dieser Module stellt 32 Ein- bzw. Ausgänge bereit. Im Projekt sind die Ports fest aufgeteilt:

- **GPIO0**: alle Eingänge, also Lichtschranken, Metallsensor, Rückmeldungen und Bedientaster
- **GPIO1**: Ausgänge für Motor, Ampel und Weiche bzw. Auswerfer
- **GPIO2**: Ausgänge für die Taster-LEDs und die Signalleuchten Q1/Q2

Tabelle 3.2.3.1: Kenndaten des BeagleBone Black

Komponente	Beschreibung
SoC	TI AM335x (Sitara), ARM Cortex-A8, bis 1 GHz
Speicher	512 MB DDR3
I/O	4 GPIO-Module
Netzwerk	10/100-Ethernet

3.2.4 Speicherabbildung und I/O -Belegung

Dieser Abschnitt dokumentiert die technische Schnittstelle zwischen Software und Hardware. Er schlüsselt die Basisadressen der GPIO-Module auf und ordnet in einer detaillierten Tabelle alle Signale, Ports und Bits den jeweiligen Sensoren, Aktoren und Tasten zu.

Tabelle 3.2.4.1: Basisadressen der GPIO-Module

Modul	Basisadresse	Nutzung
GPIO0	0x44E0_7000	Eingänge
GPIO1	0x4804_C000	Ausgänge Motor, Ampel, Weiche
GPIO2	0x481A_C000	Ausgänge Taster-LEDs, Q1/Q2

Tabelle 3.2.4.2: I/O-Belegung

Bezeichner	Port.Bit	Bedeutung
L_Anfang	GPIO0.2	Werkstück im Einlauf
H_OK	GPIO0.4	Höhe im Toleranzbereich
L_Aussortierer	GPIO0.5	Werkstück in Weiche/Auswerfer
MD	GPIO0.7	Metall erkannt

W_Offen	GPIO0.14	Weiche offen
L_Rutsche	GPIO0.15	Rutsche voll
L_Ende	GPIO0.20	Werkstück im Auslauf
K_Start	GPIO0.22	Taste Start gedrückt
K_Stop	GPIO0.23	Taste Stop gedrückt
K_Reset	GPIO0.26	Taste Reset gedrückt
E_Stop	GPIO0.27	E-Stop gedrückt
M_Rechts	GPIO1.12	Motor Rechtslauf
M_Links	GPIO1.13	Motor Linkslauf
M_Langsam	GPIO1.14	Motor langsam
M_Stop	GPIO1.15	Motor Stop
AM_Rot	GPIO1.16	Ampel rot
AM_Gelb	GPIO1.17	Ampel gelb
AM_Gruen	GPIO1.18	Ampel grün
W_A	GPIO1.19	Weiche öffnen bzw. Auswerfer ausfahren
LED_Start	GPIO2.2	LED Taste Start
LED_Reset	GPIO2.3	LED Taste Reset
LED_Q1	GPIO2.4	Signalleuchte Q1
LED_Q2	GPIO2.5	Signalleuchte Q2

3.2.5 Anforderungen aus dem Verhalten und technischen Besonderheiten

Hier werden spezifische Rahmenbedingungen und hardwarebedingte Herausforderungen der Anlage in Form von Anforderungen (HW_REQ) festgelegt. Dazu gehören unter anderem Sensorspezifika, Toleranzen bei der Motorgeschwindigkeit, Besonderheiten der Weichenansteuerung sowie Reflexionseigenschaften unterschiedlicher Werkstückematerialien.

Lfd. Nr. / ID	Beschreibung
HW_REQ_1	Der Höhenmesssensor ist angewinkelt. Die Winkel unterscheidet sich bei jeder Anlage.
HW_REQ_2	Die Geschwindigkeit des Motors variiert leicht.
HW_REQ_3	Bei manchen Anlagen handelt es sich um eine Weiche, bei anderen um einen Auswerfer.
HW_REQ_4	Die Weiche ist stromlos geschlossen. Die Spule darf nur kurz bestromt werden, sonst überhitzt sie.
HW_REQ_5	Werkstücke reflektieren evtl. unterschiedlich je nach Farbe und Material
HW_REQ_6	
HW_REQ_7	

3.3 Softwareebene

Hier wird die Software für die Anlagensteuerung spezifiziert. Auf Basis von Systemanforderungen und Prozessmerkmalen werden die konkreten Vorgaben abgeleitet. Unterschiede zwischen den Anlagenteilen werden explizit berücksichtigt.

3.3.1 Systemkontext der Software

Dieser Abschnitt definiert die Grenzen der Software und beschreibt die Schnittstellen zu Bedienern, Sensorik, Aktorik und Nachbarsystemen.

3.3.2 Resultierende Anforderungen an die Software

Hier werden spezifische Verfeinerungen, neue Anforderungen sowie der Umgang mit Fehlbedienungen und Systemstörungen dokumentiert, ohne Selbstverständlichkeiten zu wiederholen.

Lfd. Nr. / ID	Beschreibung
< SW_REQ_x>	*<Beschreibung>*

3.3.3 Schnittstellen: Hardwareabstraktionsschicht (HAL)

In dem folgenden Abschnitt werden die Schnittstellen der HAL spezifiziert und beschrieben.

3.3.3.1. Rahmen

- Die HAL kapselt Sensoren, Aktoren, ADC und Interrupt Controller, wobei Aktoren in einem eigenen Modul gehalten werden.
- Aufrufe sind nicht blockierend, sofern nicht anders angegeben.
- Die HAL soll Aufgaben zur Datenauswertung enthalten, um die Logik zu entlasten (Auswertung der Höhenmessung).

3.3.3.2. Enums

Name	Werte
`Light Barrier`	`LB_START, LB_HEIGHT, LB_SWITCH, LB_END`
`Button`	`BTN_START, BTN_STOP, BTN_RESET, BTN_START_LONG`
`Switch`	`SWI_ESTOP`
`Color`	`RED, YELLOW, GREEN`
`Direction`	`RIGHT, LEFT`
`Speed`	`SLOW, FAST`

Name	Werte
`Light Barrier`	`LB_START`, `LB_HEIGHT`, `LB_SWITCH`, `LB_END`
`Button`	`BTN_START`, `BTN_STOP`, `BTN_RESET`, `BTN_START_LONG`
`Switch`	`SWI_ESTOP`
`Color`	`RED`, `YELLOW`, `GREEN`
`Direction`	`RIGHT`, `LEFT`
`Led`	`Q1`, `Q2`
`Ramp`	`RAMP_FULL`, `RAMP_NOTFULL`

3.3.3.3. Sensoren

Alle Funktionen lesen den aktuellen Zustand und sind nicht blockierend. Für das Auslesen der Taster (Btn_Start, Btn_Stop, Btn_Reset) gibt es keine gesonderte Funktion, da dies rein über Interrupts realisiert wird.

Funktion	Rückgabe	Beschreibung
`bool isDivOpen()`	`true` = Weiche geöffnet	Rückmeldung der Weiche

3.3.3.4. Aktoren

Für die Aktoren verwenden wir das strategy design pattern, um je nach Anlage die richtige Aussortierungs Strategie verwenden zu können.

Funktion	Beschreibung
`void beltRun(Speed s)`	Band starten (Geschwindigkeit)
`void beltPause()` / `void beltUnpause()`	Band anhalten bzw. fortsetzen (Konfiguration bleibt unverändert)
`void beltStop()`	Band stoppen (setzt Speed und Direction zurück
`void eject()`	Auswerfer auslösen
`void switchOpen(int ms)`	Weiche für `ms` Millisekunden öffnen
`void letPass()`	
`void sortOut()`	
`void stackLightOn(Color c)` / `void stackLightOff(Color c)`	Ampelfarbe ein- bzw. ausschalten
`void stackLightBlinkOn(Color c,float freq)` / `void stackLightBlinkOff(Color c)`	Ampelfarbe blinkend ei n- bzw. ausschalten (**blockierend**)
`void ledOn(Led l)` / `void ledOff(Led l)`	LED Q1/Q2 ein- bzw. ausschalten

3.3.3.5. Höhenmessung und ADC

Funktion	Beschreibung
`void heightMeasure(int ms)`	Startet die Messung für `ms` Millisekunden, **blockierend**. Nach Ablauf sendet die HAL das Ergebnis als Pulse. Wird `measure()` aufgerufen, während eine Messung läuft, wird der Aufruf ignoriert.
`int getHeight()`	Skalierter Messwert nach Kalibrierung (z. B. für Debugging)
`int getHeightRaw()`	Nicht skalierter Messwert (z.B. für die Kalibrierung)

3.3.3.6. Interrupts und Timer

Funktion	Beschreibung
`void initInterrupts(int channel)`	Initialisiert die Interrupts. Die HAL meldet Interrupts per Pulse an den angegebenen Channel.
`void initTimer()`	Initialisiert den/die Timer

3.3.3.7. Interrupt Events

Quelle	Event	Flanke	Semantik

`LB_START`	`EV_LB_START_TRIGGERED`
	
	`EV_LB_START_FREED`
		`beide`	Werkstück am Bandanfang erkannt

`LB_SWITCH`	`EV_LB_SWITCH_TRIGGERED`
	
	`EV_LB_SWITCH_FREED`
		`beide`	Werkstück am Sortierer

`LB_RAMP`	`EV_LB_RAMP_TRIGGERD`
	
	`EV_LB_RAMP_FREED`
		`beide`	Rutsche voll bzw. wieder frei

`LB_END`	`EV_LB_END_TRIGGERD`
	
	`EV_LB_RAMP_FREED`
		`beide`	Übergabe an CBM2
			Warten bis Übergabe möglich

`METAL`	`EV_METAL_DETECTED`	`steigend`	Werkstücktyp bestimmen
`BTN_START`	`EV_BTNSTART_LONGPRESSED`	`beide`	`Wechesel zu Betrieb/Service Modus basiert auf`

	EV_BTNSTART_SHORTPRESSED		die Dauer des Drucks
BTN_STOP	EV_BTN_STOP	fallende	Betrieb stoppen
BTN_RESET	EV_BTN_RESET	fallende	Fehler quittieren
ESTOP	ESTOP_PRESSED / _RELEASED	beide	Not-Halt / Freigabe
Höhen-ADC	HEIGHT_SAMPLE_READY	ADC-IRQ	Messwert liegt vor

3.3.4 Schnittstellen: Nachrichten und Signale

Dieser Abschnitt gruppiert alle ein- und ausgehenden Signale und Nachrichten übersichtlich in Tabellen. Die hier definierten Kurzbezeichner sind ab sofort im gesamten Projekt verbindlich.

Welche ein- und ausgehenden Signale/Nachrichten ergeben sich aus der Analyse der Anlage und aus den Anforderungen an das SW-System? Wie erfolgt die Kommunikation mit Nachbarsystemen? Gruppieren Sie die Nachrichten und Signale sinnvoll und teilen Sie diese in einzelne Tabellen auf.

Hinweis: Führen Sie hier schon sinnvolle und nachvollziehbare Kurzbezeichner ein, vergessen Sie aber nicht diese in den Tabellen auch entsprechend zu erläutern und zu beschreiben. Es bieten sich wiederkehrende Muster/Schemata für die Vergabe von Label und Kurzbezeichner an.

Nutzen Sie diese Bezeichner spätestens ab hier dann auch fortlaufend im Dokument und Ihrem Entwicklungsprozess!

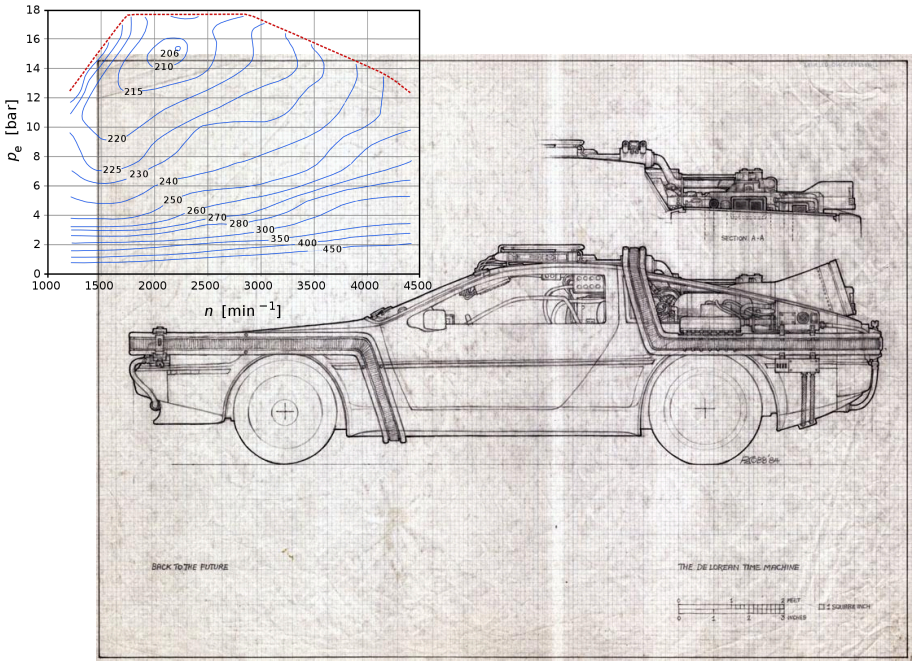
4 Grobkonzept des technischen Systementwurfes

Eine kompakte, textlich und visuell gestützte Übersicht über Systemaufbau, Grobarchitektur, Kommunikation und Besonderheiten des finalen Produkts zum schnellen Verständnis für Projektbeteiligte.

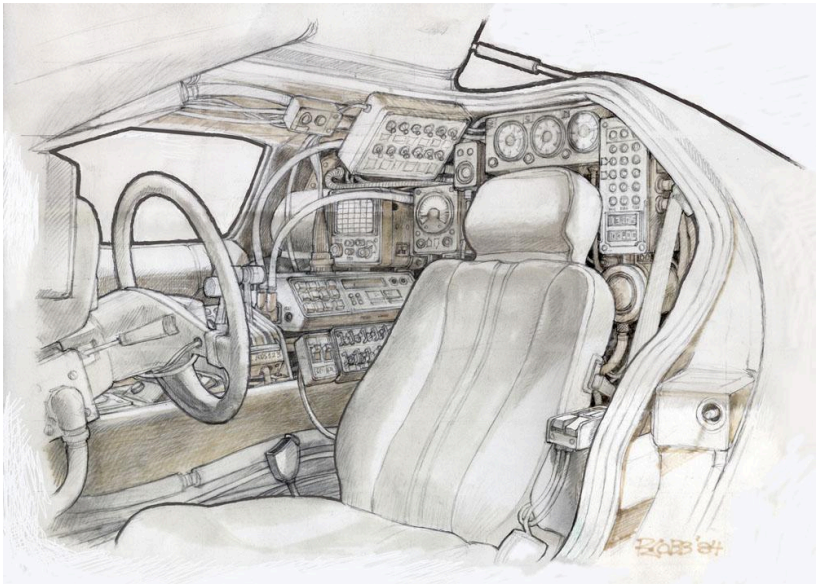
Beschreiben Sie kurz und knapp auf **zwei** Seiten, wie Ihr Konzept für das finale Produkt aussehen soll. Mögliche Aspekte sind: Systemaufbau, Grobarchitektur, System-Kommunikation, Besonderheiten, etc. Dabei bitte beachten: Bilder helfen Sachverhalte zu verdeutlichen, reichen aber nicht allein aus. Beschreiben Sie ihr Konzept daher auch textuell.

Fiktives Bsp.:
Wir bauen ein Auto. Das Auto hat Platz für 1 Person, ist ein 1-Türer und soll in der Farbe silbern angeboten werden. Wir bauen auf E-Antrieb und verzichten auf einen Kofferraum, um möglichst viel Platz für den Passagier im Innenraum zu bekommen. Das Fahrzeug wird aus einem Monocoque aufgebaut. Besonderheit: Die Zeit vergeht während der Fahrt wie im Fluge. Wir nutzen dafür …

Ziel dieser Beschreibung:
Der Leser / der Betreuer / das andere Teammitglied kann schnell verstehen, was ihr vorhabt.



Quellen: https://blog.atomlabor.de/das-back-to-future-delorean-design-von
https://de.wikipedia.org/wiki/Verbrauchskennfeld



5 Software-Design

Dieses Kapitel bildet die verbindliche Designbasis für die Implementierung. Die nachfolgenden Modelle müssen im Quellcode exakt so umgesetzt werden.

Strukturen, Verhalten und Bezeichner im Code müssen mit dem Modell übereinstimmen. Daher ist ein wohlüberlegtes Design wichtig.

Anmerkung: Die Implementierung MUSS zu Ihrem Design-Modell konsistent sein.

5.1 Software Architektur

Hier werden die übergeordneten Komponenten, deren Schnittstellen, gewählte Entwurfsmuster und zentrale technische Entscheidungen dokumentiert.

Erstellen Sie eine Architektur für Ihre Software. Geben Sie eine kurze Beschreibung der Komponenten und Schnittstellen an. Dokumentieren Sie hier wichtige technische Entscheidungen. Welche Patterns werden gegebenenfalls verwendet? Wie erfolgt die interne Kommunikation?

5.2 Software Struktur

Die Verfeinerung der Architektur durch UML-Klassendiagramme, ergänzt durch eine textuelle Beschreibung der Komponenten-Zuständigkeiten.

Der nächste Schritt ist die Verfeinerung der Architektur. Zeigen Sie die Struktur ihrer Software-Komponenten und dokumentieren Sie sie mit Hilfe von UML Klassendiagrammen, unter Beachtung der Designprinzipien. Die Modelle können mit Hilfe eines UML-Tools erstellt werden. Hier ist dann ein Übersichtsbild einzufügen.

Ergänzen Sie jeweils eine kurze textuelle Beschreibung ihrer Komponenten mit Erläuterungen der Zuständigkeiten (Responsibility), Schnittstellen usw.

5.3 Verhaltensmodellierung

Mithilfe von Verhaltensdiagrammen wird das zeitliche und zustandsbasierte Verhalten der Software auf Basis der Anforderungen abgebildet.

Ihre Software muss zur Bearbeitung der Aufgaben ein Verhalten aufweisen/abbilden. Überlegen Sie sich dieses Verhalten auf Basis der Anforderungen und modellieren Sie das Verhalten unter Verwendung von Verhaltensdiagrammen aus den Vorlesungen.

6 Implementierung: Besonderheiten

Statt den vollständigen Quelltext zu zeigen, werden hier nur architektonisch wichtige Details und ausgewählte Code-Snippets erläutert.

Anmerkung: Nur wichtige Implementierungsdetails sollen hier erklärt werden. Code-Beispiele (snippets) können hier aufgelistet werden, um der Erklärung zu dienen.

Anmerkung: Bitte KEINE ganzen Programme hierhin kopieren!

7 Qualitätssicherung

Dieses Kapitel beschreibt das Konzept zur automatisierten Verifikation und Validierung der Software über alle Teststufen hinweg – vom Unit-Test bis zur Abnahme.

7.1 Teststrategie

Dieser Abschnitt definiert den zeitlichen Ablauf der Teststufen, die Test-Architektur sowie konkrete Testmethoden *(z. B.Grenzwertanalyse, Code-Abdeckungen)* zur weitestgehenden Automatisierung der Qualitätssicherung. Wir testen in jedem Sprint die Funktionen, die neu hinzukommen. Automatisiert wird nur, was die Aufgabe verlangt oder ohne das System einfach prüfbar ist.

Teststufe	**Was**	**Wie**	**ab**
Integrationstest	Qnet zwischen CBM1 und CBM2	Testprogramm prüft die Datenübertragung	Termin 1
Komponententest	Zustandsautomaten	Ereignisfolgen über einen HAL Mock einspeisen, automatisiert, 100 % Zustandsüberdeckung	Termin 3
Systemtest	ein CBM, dann das System	Testfälle aus 7.2 an der CBM	Termin 4
Abnahmetest	System	Testfälle aus 7.2	Termin 6

Für jeden Zustandsautomaten gibt es Ereignisfolgen, die zusammen jeden Zustand mindestens einmal erreichen. Abnahmetests leiten wir aus den User Stories ab (Kap. 3.1.4). Die automatisierten Tests laufen vor jedem Merge, das ist bereits Teil der Definition of Done (Kap. 1.3). Reviews sind in Kap. 1.3 und 2.4 geregelt.

7.2 Testszenarien/Abnahmetest

Hier werden die aus den Kundenanforderungen abgeleiteten Blackbox-Testfälle auf Systemebene in Tabellenform dokumentiert, inklusive Vorbedingungen, Testschritten und erwarteten Ergebnissen.

AT_01	**E-Stop an CBM1 und Wiederanlauf**
Requirements	(81) bis (84), US_38, US_39, US_40, US_41
Kurzbeschreibung	Ein E-Stop an CBM1 hält das ganze System an. Das System läuft erst nach Lösen des E-Stops und Reset an beiden CBM wieder an.
Vorbedingungen	CBM1 und CBM2 im Betriebszustand, kein Fehler, keine Warnung, ein Werkstück liegt auf CBM1 vor der Höhenmessung

Schritt	**Aktion**	**Erwartung**	**Erfüllt**
1	E-Stop an CBM1 drücken	Beide Conveyor Belter stehen sofort. Weiche bzw. Auswerfer sind stromlos. Beide Konsolen melden im Klartext den E-Stop an CBM1.	
2	Reset an CBM1 und CBM2 drücken, E-Stop bleibt gedrückt	Keine Reaktion, beide Conveyor Belter stehen	
3	E-Stop an CBM1 lösen	System bleibt stehen	
4	Reset nur an CBM1 drücken	System bleibt stehen	
5	Reset an CBM2 drücken	E-Stop Meldung verschwindet auf beiden Konsolen, System wechselt in den abgestimmten Folgezustand	
6	Conveyor Belter prüfen	Kein Werkstück ist heruntergefallen (32)	

7.3 Testprotokolle und Auswertungen

Dokumentation der historischen und aktuellen Testprotokolle, inklusive des finalen Abnahmeprotokolls mit erfolgreich nachgewiesenen Funktionen und einer Mängelliste. Pro Testlauf an der Anlage halten wir fest: Test ID, Stand (Tag), Datum, Tester, Ergebnis, Beobachtung. Bei Fehlschlag wird ein Issue mit Label bug angelegt (Kap. 1.3). Das letzte Protokoll ist das Abnahmeprotokoll.

8 Technische Schulden

Übersicht über bekannte technische Mängel, offene Optimierungspotenziale und grobe Aufwandsabschätzungen für deren spätere Beseitigung, um Transparenz gegenüber dem Kunden zu schaffen.

9 Lessons Learned

Ergebnisse der teaminternen Reflexion mit jeweils mindestens drei Punkten zu positiven Aspekten,Verbesserungspotenzialen und konkreten Learnings für zukünftige Projekte.

Führen Sie ein Teammeeting durch, in dem gesammelt wird, was gut gelaufen war, was schlecht gelaufen war und was man im nächsten Projekt (z.B. im PO) besser machen muss und will. Listen Sie für die Aspekte jeweils mindestens drei Punkte auf. Weitere Erfahrungen und Erkenntnisse können hier ebenso kommentiert werden, auch Anregungen für die Weiterentwicklung des Praktikums.

10 Anhang

10.1 Glossar

Alphabetisch sortierte und eindeutige Erklärungen aller zentralen Fachbegriffe und Abkürzungen, die im Rahmen der Dokumentation verwendet werden.

Begriff	Definition
Conveyor Belt Module / CBM	Eine der beiden baugleichen Festo-Transfereinheiten. Sie besteht aus Conveyor Belt, Sensorik, Aktorik, Bedienpanel und einem eigenen Steuerungsrechner
System	Die Werkstück-Sortieranlage als Ganzes. Sie besteht aus den beiden Conveyor Belt Modules CBM 1 und CBM 2.
Betriebszustand	Zustand, in dem die Anlage Werkstücke annimmt und sortiert. Er wird über die Taste Start erreicht . Die Ampel leuchtet dann grün .
Ruhezustand	Zustand, in dem die Anlage eingeschaltet ist, aber keine Werkstücke sortiert. Er wird über die Taste Stop erreicht .
Service-Mode	die Modus, die durch langes Drücken der Taste Start erreicht wird. In ihr laufen Kalibrierungen und Selbsttests . Die grüne Ampel blinkt.
Steuerungsrechner	embedded-Rechner (BeagleBone Black), der genau ein Conveyor Belt Module steuert.
Werkstück-ID	Eindeutige Kennung, die die Software einem Werkstück vergibt, sobald es am Anfang von CBM1 erkannt wird
Werkstücktyp	Klasse eines Werkstücks, die sich aus den gemessenen Merkmalen ergibt (Höhe, Bohrung, Metall). Sie ist die Grundlage für die Sortierentscheidung.
Bedienfeld	Bedienpanel eines Conveyor Belt Modules, über das der Bediener die Anlage steuert und Statusinformationen erhält. Es umfasst die Tasten Start, Stop und Reset, den E-Stop-Schalter, die LEDs in den Tasten Start und Reset (LED_Start, LED_Reset) sowie die Signalleuchten Q1 und Q2 (LED_Q1,

	LED_Q2)
Werkstück erkennen	Feststellen, dass sich an einer bestimmten Position der Förderstrecke ein Werkstück befindet. Grundlage ist der Signalwechsel eines Sensors, z. B. die Unterbrechung der Lichtschranke L_Anfang. Das Erkennen sagt nichts über den Typ des Werkstücks aus. Wird ein Werkstück am Anfang von CBM1 erkannt, vergibt die Software seine Werkstück-ID
Werkstück klassifizieren	Ordnet einem bereits erkannten Werkstück anhand seiner gemessenen Merkmale (Höhenprofil, Metall) einen Werkstücktyp zu. Werkstücke, die für die Sortierreihenfolge oder das System nicht relevant sind werden als “falsches Werkstück” beziehungsweise “Sonderform” erkannt.
Werkstück, sortierbar	Werkstück, das die Anlage ohne Eingriff des Bedieners verarbeiten kann. Sie transportiert es, klassifiziert es und gibt es entweder in der Sortierreihenfolge an das Ende von CBM2 weiter oder sortiert es in eine Rutsche aus. Dazu gehören die Standardwerkstücke und die defekten Werkstücke .
falsches Werkstück	Sortierbares Werkstück mit der Grundform eines Standardwerkstücks , dessen gemessene Merkmale keinem bekannten Werkstücktyp entsprechen. Es wird immer aussortiert .
Werkstück, Sonderform	Werkstück, dessen Geometrie von der Grundform der Standardwerkstücke abweicht und das die Anlage deshalb nicht sicher transportieren oder aussortieren kann. Der Bediener muss es vom Band entfernen.
Sicherer Zustand	Zustand des Systems, in den es wechselt, sobald an einem der beiden CBMs der E-Stop betätigt wird . In diesem Zustand steht an beiden CBMs der Motor (M_Stop), Weiche bzw. Auswerfer sind stromlos , und kein Aktor wird bewegt. An dem Conveyor Belt Module, an dem der E-Stop ausgelöst wurde, wird das signalisiert . Das System verlässt den sicheren Zustand erst, wenn der E-Stop gelöst und an beiden CBMs Reset gedrückt wurde .
eingreifen von Betreiber	Manuelle Handlung des Betreibers direkt an der Förderstrecke des Systems ?
Werkstückübergabe	Vorgang innerhalb des Systems, bei dem ein Werkstück vom Ende von CBM1 auf den Anfang von CBM2 wechselt. Er besteht aus zwei Teilen. Physisch:

	Das Werkstück wird vereinzelt übergeben , und nur, wenn CBM2 bereit ist (höchstens zwei Werkstücke auf CBM2, mindestens 25 cm Abstand). Daten: Werkstück-ID und Typ werden über Qnet an CBM2 übertragen. Die Übergabe beginnt, wenn das Werkstück L_Ende von CBM1 verlässt, und endet, wenn es L_Anfang von CBM2 erreicht.
Werkstückprüfung	Abgleich auf CBM2: Die dort gemessenen Merkmale eines Werkstücks (Werkstücktyp, Höhe) werden mit den Daten verglichen, die CBM1 bei der Werkstückübergabe übermittelt hat. Stimmen sie nicht überein, wird das Werkstück aussortiert
Werkstück	Physisches Objekt welches von der Anlage sortiert werden soll
Standard Werkstück	Werkstücktyp welcher der Anlage bekannt ist. Vgl. Kapitel Werkstücke
Quittierungsprozess	Ablauf der beim Auftreten eines Fehlers durchzuführen ist. vgl. Kapitel Fehlerbehandlung

10.2 Abkürzungen

Listen Sie alle wichtigen Abkürzungen auf, die Sie in diesem Dokument benutzt haben. Wählen Sie eine sinnvolle Unterteilung (Signale/Events, Hardwarebezeichner etc.). Nutzen Sie eine alphabetische Sortierung!

AM - Amirhossein Naghashi

AS - Alexander Freiherr von Seld

MD - Mohammad Danandeh

EP - Emil Peters

CBM_1 = Conveyor Belt Module 1

CB = Conveyor Belt

CB_Start(_1) = Conveyor Start (CBM1)

CB_End = Conveyor End

LB_Start(_1) = Light Barrier Start (CBM_1)

LB_M = Light Barrier Middle/Metall

LB_End = Light Barrier End

LB_R = Light Barrier Reject Ramp(Rutsche)

HS_1 = Height Sensor (CBM_1)

MS_1 = Metall Sensor (CBM_1)

LED_Q1

LED_Q2

SL_Red_1 = Stack Light (Ampel)

SL_Yellow

SL_Green

Ramp

Btn_Start = Button Start

Btn_Stop = Button Stop

Btn_Reset = Button Reset

SWI_EStop_1 = Switch E-Stop CBM1

Push = Pusher (Auswerfer)

Div = Diverter (Weiche)

WP = Workpiece

WP_A , WP_B, WP_C, WP_D ?

WP_Wrong = Falsches Werkstück (auch defekt oder unbekannt)

WP_Special = Sonderform Werkstück

Idle_Mode = Ruhezustand

Running_Mode = Betriebszustand

Service_Mode

Error_Mode (oder Fault_Mode ?)

EStop_Mode